

21

Machine learning

Syllabus objectives

- 5.1 Explain and apply elementary principles of machine learning.
 - 5.1.1 Explain the main branches of machine learning and describe examples of the types of problems typically addressed by machine learning.
 - 5.1.2 Explain and apply high-level concepts relevant to learning from data.
 - 5.1.3 Describe and give examples of key supervised and unsupervised machine learning techniques, explaining the difference between regression and classification and between generative and discriminative models.
 - 5.1.4 Explain in detail and use appropriate software to apply machine learning techniques (*eg* penalised regression and decision trees) to simple problems.
 - 5.1.5 Demonstrate an understanding of the perspective of statisticians, data scientists, and other quantitative researchers from non-actuarial backgrounds.

0 Introduction

The aim of this chapter is to provide an insight into the topic of machine learning. Machine learning is a vast topic and this chapter will only provide a high-level introduction. Specifically, the chapter has the following aims:

- To provide a high-level knowledge of the various branches of machine learning and examples of their applications, both within general industry and within the specific sectors that actuarial work involves. The level of knowledge targeted is such as will allow you to identify whether any branch of machine learning would be useful in addressing any problem you face.
- To provide you with sufficient background information that you can participate in high-level conversations related to projects involving machine learning analyses and their results.
- To describe some of the most common machine learning techniques.
- To discuss the relationship between machine learning and other branches of data science and statistical analysis, so that you are able to communicate effectively with other quantitative researchers, and to understand the similarities and differences between machine learning and other approaches.

There are many resources available to students to gain an insight into the key elements of machine learning. One excellent resource is a series of lectures given at Caltech by Yaser Abu-Mostafa which is freely available online at <https://work.caltech.edu/telecourse.html>.

Another is A. Chalk and C. McMurtrie 'A practical introduction to Machine Learning concepts for actuaries', *Casualty Actuarial Society E-forum*, Spring 2016.

1 What is machine learning?

Machine learning describes a set of methods by which computer algorithms are developed and applied to data to generate information. This information can consist simply of hidden patterns in the data, but often the information is applied to solve a specific problem.

Machine learning methods have become popular in recent years with the advent of increasing quantities of data and the concomitant rapid increase in computing power.

When describing the new age of 'big data' researchers often talk about the three V's: volume, velocity and variety. There are now very large *volumes* of data in use, which computers can process very rapidly (*velocity*) and the data can take many different forms (*variety*).

In order for machine learning to be useful in tackling a problem we need the following to apply:

- **A pattern should exist. If there is no pattern, there is no information to be had, and machine learning will not help (indeed, it might be counterproductive by 'discovering' patterns that do not exist).**
- **The pattern cannot be practically pinned down mathematically by classical methods. If it could be pinned down, we could proceed to describe it mathematically.**
- **We have data relevant to the pattern.**

Examples of problems which are commonly solved in this way include:

- **targeting of advertising at consumers using web sites**
- **location of stock within supermarkets to maximise turnover**
- **forecasting of election results**
- **prediction of which borrowers are most likely to default on a loan.**



Question

Give some other examples where machine learning is used:

- (a) in everyday life
- (b) in an actuarial / insurance / financial context.

Solution

(a) Other everyday examples include:

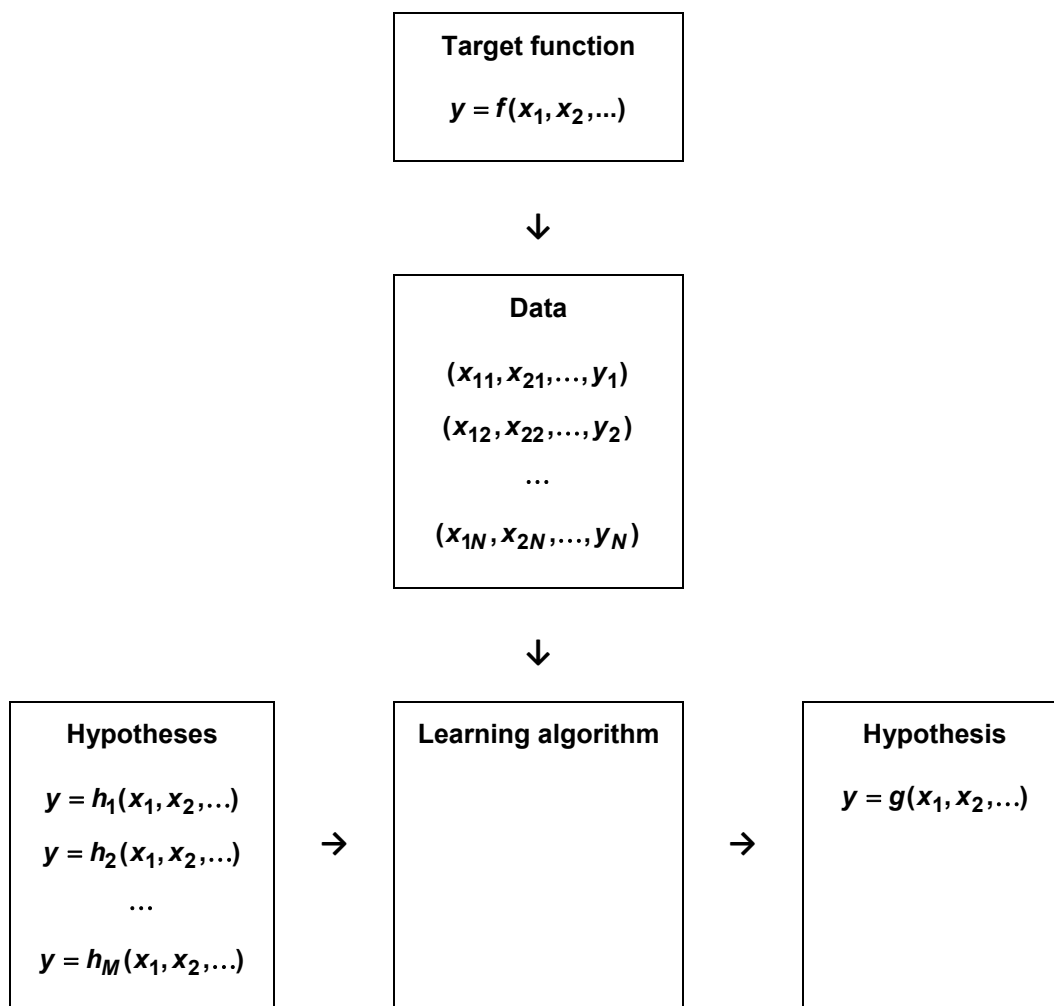
- using an internet search engine such as Google to find relevant web pages
- using a spam filter to identify and remove unwanted emails
- using face recognition to identify known criminals on CCTV footage
- identifying criminals using fingerprints or DNA
- recommending items to purchase on online shopping sites
- matching job applicants to available positions
- suggesting suitable matches on a dating app
- finding relevant historical records on family tree websites
- classifying people in your photos automatically by name
- recognising voice commands, eg in apps such as Siri, Cortana and Alexa
- converting handwriting to text
- translating text from one language to another.

(b) Other examples in an actuarial / insurance / financial context include:

- classifying the risk for motor insurance policyholders using in-car monitoring devices
 - identifying marker genes that are associated with particular medical conditions
 - identifying insurance claims that might be fraudulent
 - identifying fraudulent benefit claims
 - identifying fraudulent tax declarations.
-

2 An overview of machine learning

The diagram below (due to Yaser Abu-Mostafa) provides an overview of the machine learning process.



First, there is some target function f , which maps a set of variables, or *features*, that we can measure, onto some output y . (What we term ‘variables’ or ‘covariates’ in statistical modelling, machine learning terms ‘features’).

We will assume that we have identified J features that we consider relevant.

Let the variables, or features, be $x_1, x_2, \dots, x_J, \dots, x_J$.

Then we have:

$$y = f(x_1, x_2, \dots, x_J)$$

The target function is unknown and it is this which we are trying to approximate. The target function might, for example, map life insurance data such as smoking behaviour, lifestyle factors and parental survival to life expectancy.

Second, we have data on y and $x_1, x_2, \dots, x_J$ for a sample of N individuals.

We use the data to develop a hypothesis which relates the data to the output. Let the hypothesis be:

$$y = g(x_1, x_2, \dots, x_J)$$

The idea is that $g(x_1, x_2, \dots)$ should be close to the unknown function $f(x_1, x_2, \dots)$.

The way the hypothesis $y = g(x_1, x_2, \dots)$ is chosen is by trying out a large number, say M , of hypotheses $y = h_1(x_1, x_2, \dots)$, $y = h_2(x_1, x_2, \dots)$, ..., $y = h_M(x_1, x_2, \dots)$ on the data and using a learning algorithm to choose among them. The hypotheses are usually drawn from a *hypothesis set*, which has a general form.

So, for example, in classical linear modelling the hypothesis set might be the set of linear relationships:

$$y = w_{10} + w_{11}x_1 + w_{12}x_2 + \dots + w_{1j}x_j + \dots + w_{1J}x_J$$

$$y = w_{20} + w_{21}x_1 + w_{22}x_2 + \dots + w_{2j}x_j + \dots + w_{2J}x_J$$

...

$$y = w_{m0} + w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mj}x_j + \dots + w_{mJ}x_J$$

...

$$y = w_{M0} + w_{M1}x_1 + w_{M2}x_2 + \dots + w_{Mj}x_j + \dots + w_{MJ}x_J$$

where the w_{mj} are weights to be applied to the features. There are M hypotheses, each with a different set of weights.

Linear regression (which is covered in Subject CS1) can be viewed within this framework. The weights are equivalent to regression coefficients and the final hypothesis $y = g(x_1, x_2, \dots)$ is the set of weights which 'best fits' the data according to some criterion, such as minimising the squared distance between the values of y predicted by the model and the values of y observed in reality. Of course, the linear regression problem is typically solved in 'one step', whereas many machine learning problems are solved iteratively, or in many steps.

3 Concepts in machine learning

An important difference between machine learning and many statistical applications is that the goal of machine learning is to find an algorithm that can predict the outcome y in previously unseen cases.

In the example studied by Chalk and McMurtrie, the task was to predict the cause codes of aviation accidents from the words in brief narratives of the accidents.

This refers to a case study in the paper mentioned in the Introduction.

The cause codes in their example were 'aircraft', 'personnel issues', 'environmental issues' and 'organisational issues'. The idea of classifying insurance claims in this way has wide actuarial applications – for example, in the construction of different pricing models for different types of claim. But if an insurer uses such cause codes, a change in the IT system or in the staff that handle claims could result in claims not being coded or being coded inaccurately.

This is because the existing claims team will have established a 'rule book' with conventions that tell them which is the best code to record the accidents under when the cause is not clear-cut.

Doctors face a similar problem when recording the cause of death on a patient's death certificate. If an elderly patient dies during an operation to treat a cancer they were suffering from, this could be recorded as 'old age', 'cancer' or 'complications during surgery'.

It would be useful to develop a way of using narrative descriptions of claims to add cause codes to those for which codes are not available, so that continuity of coding could be maintained. We might do this by creating an algorithm which uses the claims narratives from data that were cause-coded to work out the cause codes that were given, and then apply this algorithm to claims *that were not cause-coded*.



Question

Suppose that we wish to use a similar system to identify cause codes for motor claims based on the descriptions policyholders write on their claim forms.

- (i) List some cause codes that we might wish the system to identify.
- (ii) List some keywords that the final algorithm might search for in the inputs.

Solution

- (i) The cause codes would probably include the familiar third party damage, fire and theft, as well as others such as own vehicle damage and personal injury.
- (ii) There would be a large number of relevant keywords in the policyholders' descriptions, eg 'crashed', 'hit', 'stolen', 'ran over', 'brakes', 'suddenly', 'tree', 'dog', 'ice' and 'exploded'.

Some of these keywords map in an obvious way to a cause (eg 'stolen' to 'theft') but others (eg 'suddenly') are much more open-ended.

A key element of this scenario is that we are going to apply the results of the exercise to data that were not used to develop the algorithm. This means that we are interested in the performance of the algorithm not just in the sample of N cases in our data, but 'out of sample'.

'Out of sample' refers to new data outside our original sample.

This is not always the case in statistical modelling (where we are often content with the model which 'fits' our data best).

For example, when we are graduating a set of crude mortality rates, we usually restrict our results to the same range of ages for which we have data.

3.1 The loss function

One way to evaluate a hypothesis is to calculate the predictions it makes and to penalise each incorrect prediction by some loss. For example, if the prediction involves the classification of something into categories, we could say that each incorrectly classified case incurs a loss of one. We then choose the hypothesis $y = g(x_1, x_2, \dots)$ by minimising the loss function.

In this example the loss function we're using to penalise errors is the zero-one loss function, which assumes that all errors are equally bad. (The zero-one loss function is covered in the Bayesian statistics chapter in Subject CS1.)

It can be shown that for some common algorithms (such as logistic regression) maximising the likelihood is equivalent to minimising the loss function.

3.2 Model evaluation

When we fit statistical models to data, we have a range of criteria to allow us to choose the 'best' model from among a set of models (as we saw in Subject CS1).

For example, we saw that the parameters of a model can be estimated using the method of maximum likelihood or the method of moments, and that we can test the goodness of fit of a model using a chi-square test.

But evaluating a predictive model involves more than this. Even the 'best' model may not be a very good predictive model. And even if it is good, it might take a very long time to find the correct parameters, or it might be very difficult to interpret (and explain to clients).

Model evaluation therefore involves more than just applying some statistical criteria of 'fit'. We illustrate some possible measures using a model designed for classification.

Accuracy. This is the proportion of predictions that the model gets right. Usually we compare this proportion with the proportion predicted by a *naïve classifier* (eg a classifier that puts every case into the same category).

Precision and recall. Consider a diagnostic test for a medical condition. The patients who take the test either have the condition or they do not. The test will classify (predict) patients as having the condition or not according to whether the outcome of the test fulfils certain criteria.

An example of a naïve classifier in this situation is one that automatically says that the patient does not have the condition. This classifier has no ability to distinguish correctly between patients with and without the condition.

We now consider the possible outcomes of the model.

There are four possibilities, shown in the table below. This table is known as a *confusion matrix*.

The terminology is based on the idea that the matrix of possible outcomes quantifies the extent to which the test ‘confuses’ patients who do / do not have the condition.

		Test result classified / predicts patient as having condition	
		YES	NO
Patient actually has condition	YES	True positive (TP)	False negative (FN)
	NO	False positive (FP)	True negative (TN)



Question

A country is introducing a new screening programme for early identification of people with a particular type of cancer.

- (i) Explain what ‘false positive’ and ‘false negative’ results would be in this context.
- (ii) Discuss the impact of false positives and false negatives from the point of view of a patient.
- (iii) State an additional concern regarding false negatives if this had been a test for an infectious disease.

Solution

- (i) A false positive is a patient that the test flags as having the disease, but in fact does not.
A false negative is a patient that the test indicates as not having the disease, when in fact they do have it.
- (ii) A false positive outcome is undesirable because the patient may be caused unnecessary worry or required to undergo further tests or treatment before it is established that they do not actually have the disease.
A false negative outcome is also undesirable because the patient may not now be identified early enough to receive effective treatment for the disease.

- (iii) With an infectious disease, there is the additional concern that false negative patients may unknowingly spread the disease to other people.

There are several useful measures we can calculate from the confusion matrix to gauge the effectiveness of the test.

Precision is the percentage of cases classified as positive that are, in fact, positive. Using the abbreviations in the table this is:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Ideally we would like to have $FP = 0$, which would result in a precision of 1, ie 100%.

Recall is the percentage of positives that we managed to identify (correctly):

$$\text{Recall} = \frac{TP}{TP + FN}$$

This measure is also called the *sensitivity*. If $FN = 0$, ie if the test has not missed anyone who has the condition, it will equal 100%.

These can be combined in a single measure known as the F_1 score:

$$F_1 \text{ score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The 'F' here arose historically and doesn't actually stand for anything. The '1' subscript just identifies this measure out of several similar measures that have also been proposed and could be used instead.



Question

- (i) Derive and simplify a formula for the harmonic mean of the precision and the recall.

Hint: The harmonic mean of a set of values is the reciprocal of the arithmetic mean of their reciprocals.

- (ii) Comment on the answer in (i).

Solution

- (i) Using the hint given, we can see that the harmonic mean H of the precision and the recall can be found from the equation:

$$\frac{1}{H} = \frac{1}{2} \left(\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}} \right)$$

$$\text{So: } \frac{1}{H} = \frac{1}{2} \left(\frac{\text{Recall} + \text{Precision}}{\text{Precision} \times \text{Recall}} \right) \Rightarrow H = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Recall} + \text{Precision}} = F_1$$

- (ii) F_1 is the harmonic mean of the precision and the recall. This is different from the more familiar *arithmetic* mean, but it also gives an average value of the two measures taken together and results in a value in the same range, *ie* 0 to 1.

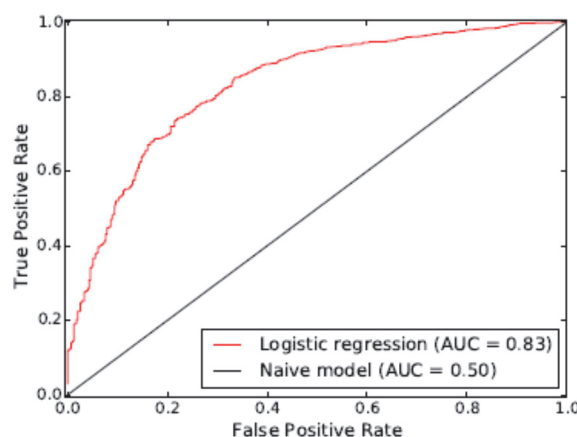
Another measure is the *false positive rate*.

There is a trade-off between the recall (the true positive rate) and the false positive rate (the percentage of cases which are not positives, but which are classified as such). The false positive rate is:

$$\text{False positive rate} = \frac{\text{FP}}{\text{TN} + \text{FP}}$$

This is not the same as $1 - \text{the true positive (ie the recall) rate}$.

Receiver operating characteristic curve. The trade-off between recall and the false positive rate can be illustrated using a *receiver operating characteristic* (ROC) curve. An example is shown below, taken from Alan Chalk and Conan McMurtrie 'A practical introduction to Machine Learning concepts for actuaries' *Casualty Actuarial Society E-forum*, Spring 2016.



This figure compares the ROC curves for a logistic regression model fitted to the cause codes for aircraft accidents with a naïve model based on random guesswork. The area under the ROC provides another single-figure measure of the efficacy of the model. The further away from the diagonal is the ROC, the greater the area under the curve and the better the model is at correctly classifying the cases.

This type of graph is most useful when the test involves a threshold of some kind. For example, a medical test may involve measuring the concentration of a particular chemical in the patient's blood and labelling the patient as positive if this exceeds a particular level. We can then calculate the false positive rate (defined above) and the true positive rate (*ie* the recall rate) for different levels of the threshold and plot these on a graph.

Points near the top left of the graph correspond to a good test where the true positive rate is high and the false positive rate is low. The diagonal line corresponds to a neutral 'zero-sum' test where there is a simple trade-off with any improvement in the true positive rate being matched by an equal deterioration in the false positive rate. The area of the triangle below the diagonal is 0.5 and the area of the whole rectangle is 1 (the maximum possible score for the ROC).



Question

Two different tests have been applied to a sample of 100 individuals to identify whether or not a particular feature is present. These resulted in the following confusion matrices:

(a) **Test 1**

PREDICTED \ ACTUAL	YES	NO	TOTAL
YES	TP = 76	FN = 4	80
NO	FP = 4	TN = 16	20
TOTAL	80	20	100

(b) **Test 2**

PREDICTED \ ACTUAL	YES	NO	TOTAL
YES	TP = 5	FN = 5	10
NO	FP = 15	TN = 75	90
TOTAL	20	80	100

Calculate the precision, recall, F_1 score and false positive rate for each matrix and comment on the answers.

Solution

$$(a) \quad \text{Precision} = \frac{TP}{TP + FP} = \frac{76}{76 + 4} = 95\%, \quad \text{Recall} = \frac{TP}{TP + FN} = \frac{76}{76 + 4} = 95\%$$

$$F_1 \text{ score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 95\% \times 95\%}{95\% + 95\%} = 95\%$$

$$\text{False positive rate} = \frac{FP}{TN + FP} = \frac{4}{16 + 4} = 20\%$$

$$(b) \quad \text{Precision} = \frac{TP}{TP + FP} = \frac{5}{5 + 15} = 25\%, \quad \text{Recall} = \frac{TP}{TP + FN} = \frac{5}{5 + 5} = 50\%$$

$$F_1 \text{ score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 25\% \times 50\%}{25\% + 50\%} = 33\%$$

$$\text{False positive rate} = \frac{FP}{TN + FP} = \frac{15}{75 + 15} = 17\%$$

The precision values show that Test 1 is much more effective at correctly identifying individuals who do have the feature.

The recall values also show that Test 1 is much better at identifying individuals who do have the feature.

The F_1 scores show that the overall performance of Test 1 is much better than Test 2.

The false positive rates are quite low for both tests, indicating that only a small proportion of individuals who do not have the feature are incorrectly flagged as having it.

3.3 Generalisation error and model validation

The methods described in the previous section allow the assessment of model performance on existing data. But how can we assess the likely predictive performance of the model? Can we be sure that we can use machine learning to test numerous hypotheses and eventually pick one which will generalise acceptably to new data? The answer is that we can in theory (see Lectures 4-6 of Yaser Abu-Mostafa's course for a demonstration and proof of this). Specifically, we can show that if the in-sample error is $E_{in}(g)$ and the out-of-sample error is $E_{out}(g)$, then:

$$P[|E_{in}(g) - E_{out}(g)| > \varepsilon] \leq 4[H(N)]e^{-\frac{1}{8}\varepsilon^2 N}$$

where:

- N is the sample size
- ε is some specified tolerance
- $H(N)$ is a polynomial in N which depends on the hypothesis set.

This equation, called the Vapnik-Chervonenkis inequality, shows that, for large enough N , it will always be possible to use learning to choose a hypothesis g which will make the tolerance as small as we like.

In this inequality, $E_{in}(g)$ and $E_{out}(g)$ are some suitable measure of the error in the results, eg the average difference between the predicted and true values, or the proportion of records that are classified incorrectly. In a good model the difference between these two quantities will be small.

For large values of N the exponential factor $e^{-\frac{1}{8}\varepsilon^2 N}$ on the RHS of the inequality will dominate the polynomial $H(N)$. So the RHS provides an upper bound that tends to zero as we increase the sample size N . This means that we can make the probability of an error of a given size as small as we like by using a big enough sample size for the training set.

This may be true in theory, but how do we test the performance of our model out-of-sample?

3.4 Train-validation-test

The conventional approach in machine learning is to divide the data into two. One part of the data (usually the majority) is used to *train* the algorithm to choose the ‘best’ hypothesis from among the M competing ones. The other is used to *test* the chosen hypothesis g on data that the algorithm has not seen before.

In practice, the ‘training’ data is often split into a part used to estimate the parameters of the model, and a part used to validate the model.

This approach is often called the train-validation-test approach. It involves three data sets:

- a **training data set**: the sample of data used to fit the model
- a **validation data set**: the sample of data used to provide an unbiased evaluation of model fit on the training dataset while tuning model hyper-parameters (see below)
- a **test data set**: the sample of data used to provide an unbiased evaluation of the final model fit on the training data set.

3.5 Parameters and hyper-parameters

In statistical analysis, we often fit models to data, for example regression models such as:

$$y_i = \beta_0 + \beta_1 x_{1i} + \dots + \beta_J x_{Ji} + \varepsilon_i \quad \text{where } \varepsilon_i \sim \text{Normal}(0, \sigma^2)$$

Here the β 's and σ^2 are the *parameters* of the model. Most supervised machine learning algorithms involve models with similar parameters. The ‘best’ values for these parameters are estimated from the data.

We will explain the difference between supervised and unsupervised learning when we look at the different branches of machine learning in Section 4.

Parameters are required by the model when making predictions. They define the skill of the model when applied to your problem and they are estimated or learned from the data. They form an integral part of the learned model.

We can say that the *parameters* of a model are variables internal to the model whose values are estimated from the data and are used to calculate predictions using the model.



Question

The number of road accidents each day is to be modelled using a linear model based on the average number of cars on the road each day. Identify the parameters in this model and describe the role they play.

Solution

If we let x denote the average number of cars on the road on a given day and y denote the number of road accidents the same day, then a linear model would take the form:

$$y = \alpha + \beta x + \varepsilon$$

In this equation, ε is an error term with mean 0, and α and β are the parameters. The values of α and β would be estimated from the data in the usual way, ie $\hat{\beta} = s_{xy} / s_{xx}$ and $\hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}$ (see page 24 of the *Tables*). The model incorporating these estimated values of the parameters would then be used to estimate values of y in the future, based on the value of x on that day, using the formula $\hat{y} = \hat{\alpha} + \hat{\beta}x$.

Machine learning algorithms, both supervised and unsupervised, however, also have higher-level attributes which must also be estimated or (in some sense) optimised. These might include:

- **the number of covariates J to include in a regression model**
- **the number of categories in a classification exercise**
- **the rate at which the model should learn from the data.**

We can speed up the learning process by allowing the machine to make quite big changes at each stage, based on the data it has just processed. However, this may result in the final model ‘overshooting’ the optimal solution or overfitting the model to the particular training set used.

A slower rate of learning will be more likely to come up with a good solution, but this will take longer to achieve.

These attributes are called *hyper-parameters*. They cannot be estimated from the data – indeed they must often be defined before an algorithm can be implemented. Hyper-parameters are external to the model and their values cannot be estimated from the data. They are typically specified by the practitioner and may be set using heuristic guidelines. Nevertheless, they are critical to the predictive success of a model.

Hyper-parameters are variables external to the model whose values are set in advance by the user. They are chosen based on the user’s knowledge and experience in order to produce a model that works well.

‘Heuristic’ means that there are no hard and fast rules for these. They are determined using rough guidelines and past experience of what works well, combined with experimentation.



Question

Give some other examples of hyper-parameters from other models that may be relevant to actuaries.

Solution

Life insurance

If we are calculating premiums for life insurance policies, we need to decide exactly how to define smoker categories such as Non-smoker, Light smoker and Heavy smoker, *eg* a person who smokes more than 20 cigarettes in an average day may be classed as a heavy smoker.

Graduation

If we are graduating mortality data using a Gompertz-Makeham formula $\mu_x = p_1(t) + \exp[p_2(t)]$,

where $p_1(t) = \sum_{k=0}^{r-1} a_k t^k$ and $p_2(t) = \sum_{k=0}^{s-1} b_k t^k$ are polynomial functions (see page 32 of the *Tables*),

we need to decide on the values to use for r and s , which determine the order of the two polynomials.

Time series

If we are fitting a linear time series using an ARIMA model, we need to decide on the values of d , p and q , which determine the number of levels of differencing to apply and the number of moving average and autoregressive terms to include.

GLM

If we are applying a generalised linear model, we need to decide on the form of the link function to use.

Reinsurance

If we are modelling large claims in general insurance, we need to specify the cut-off point for a claim to count as 'large'.

Motor insurance

If we are using geographical area as a rating factor in motor insurance, we need to decide on how many areas to use and which locations these cover.

Health

If we are using a patient's body mass index (BMI) as a predictor for the outcome of a medical procedure, we may need to specify the dividing lines between weight bands such as underweight (<20), normal (20–25), overweight (25–30) and obese (30+).

3.6 Validation and over-fitting

In a normal linear regression model, as we include more variables, the proportion of the variance in the dependent variable that is explained cannot decrease. A model with more variables will, in that sense, ‘fit’ the data better than one with fewer variables. The same is true with machine learning models, but the number of parameters in machine learning models can be very large.

There is a risk that, if the number of parameters / features is large, the estimates of the parameters in the model g that is chosen will reflect idiosyncratic characteristics of the specific data set we have used to ‘train’ the model, rather than the underlying relationships between the output, y , and the features $x_1, x_2, \dots, x_J$. This is known as *over-fitting* and is one of the biggest dangers faced by machine learning. Over-fitting leads to the identification of patterns that are not really there. More precisely, it leads to the identification of patterns that are specific to the training data and do not generalise to other data sets.

We saw this same issue when we considered the graduation of mortality rates by parametric formula in [Chapter 11](#). If we use a formula with too many parameters, the graduated rates will be undergraduated. They will follow the crude rates too closely, reflecting a lot of the random ‘noise’ present in the data, rather than just capturing the underlying pattern of the rates.

On the other hand, if the number of parameters / features is small, we might miss important underlying relationships.

We saw this issue too with graduation by parametric formula. If we use a formula with too few parameters, the graduated rates will be overgraduated. They will be smoothed too much and will not follow the underlying pattern of the rates closely enough, *eg* ‘smoothing over’ genuine features such as the teenage accident hump.

So there is a trade-off here, between bias – the lack of fit of the model to the training data – and variance – the tendency for the estimated parameters to reflect the specific data we use for training the model.

One way to assess how the predictive ability of the model changes as the number of parameters / features increases is to withhold a portion of the ‘training’ data and use it to *validate* models with different numbers of parameters / features J . One approach is to divide the training data into, say, s slices, and to ‘train’ the model s times, using a different slice for validation each time. This is called *s-fold cross-validation*.

Cross-validation is a technique to evaluate predictive models by partitioning the original sample into a training set to train the model, and a test set to evaluate it. In s -fold cross-validation, the original sample is randomly partitioned into s equal size subsamples.

One of the s subsamples is retained as the validation data for *testing* the model. The remaining $s - 1$ subsamples are used as *training* data. The cross-validation process is then repeated s times (these are the ‘folds’), with each of the s subsamples used exactly once as the validation data. The s results from the folds can then be compared (or averaged to produce a single prediction).

The advantage of this method is that all observations are used for both training and validation, and each observation is used for validation exactly once.

Typically, the error on the training data used to estimate the parameters decreases as J increases. But the prediction error on the validation data often decreases as J increases for small J , and reaches a minimum before increasing again as J gets larger. This suggests that models with a number of parameters / features close to the minimum might be most suitable and perform best out-of-sample.

So we might be able to get a better fit by adding extra parameters, but this won't necessarily make the predictions from the model any better when we apply it to new data.

3.7 Regularisation

How can we achieve a good balance between bias and variance? Put another way, is there a method that can use all the features to choose the final hypothesis g , but will prevent it becoming too complex so that generalisation is poor? There is, and it is called *regularisation* or *penalisation*. This approach exacts a penalty for having too many parameters. Recall that finding the 'best' values of the parameters, or feature weights, w_j in a machine learning problem involves minimising a loss function. Let the loss function be $L^*(w_1, w_2, \dots, w_J)$. Then the hypothesis g will be chosen to be the hypothesis with a set of weights which minimises $L^*(w_1, w_2, \dots, w_J)$.

The idea of regularisation, or penalisation, is to add to L^* a cost for model complexity.

One possibility is to add a term equal to $\lambda \sum_{j=1}^J w_j^2$, so that we now minimise the expression:

$$L^*(w_1, w_2, \dots, w_J) + \lambda \sum_{j=1}^J w_j^2$$

As noted earlier, since minimising the loss function is, in some models, equivalent to maximising the likelihood, minimising this expression is equivalent to maximising a penalised likelihood.

4 Branches of machine learning

Machine learning techniques can be divided into several branches, which we can refer to as supervised learning, unsupervised learning, semi-supervised learning and reinforcement learning. The difference between these lies not (as one might think) in the level of involvement of the human researcher in the development of the algorithm, or in the supervision of the machine. Instead, it lies in the extent to which the machine is given an instruction as to the end-point (or target) of the analysis.

The targets are sometimes referred to as *labels*.



Supervised and unsupervised learning

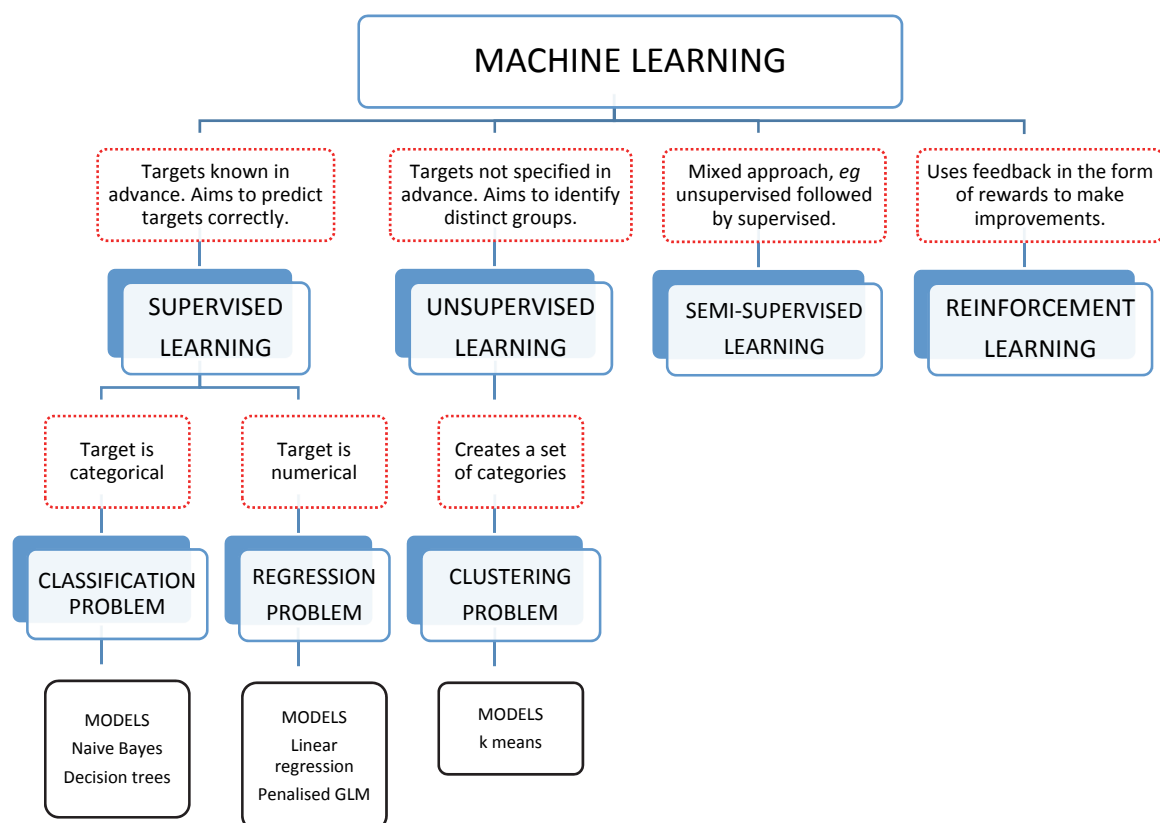
With *supervised learning*, the algorithm is given a set of specific targets to aim for.

With *unsupervised learning*, the algorithm aims to produce a set of suitable labels (ie targets).

With *semi-supervised learning*, the algorithm uses a combination of supervised and unsupervised methods.

With *reinforcement learning*, the algorithm aims to improve its performance through trial and error, using a rewards (or penalties) approach.

The diagram below shows the differences between the main branches of machine learning and the models we will consider here.





Question

- (i) Give examples of problems that would come under the headings of classification, regression and clustering.
- (ii) Give examples of problems that could be solved using semi-supervised and reinforcement learning.

Solution

- (i) ***Classification, regression and clustering***

An example of a classification problem is a spam filter that classifies emails into the two categories 'Safe' or 'Suspicious'.

An example of a regression problem is a health awareness app that predicts the user's life expectancy.

An example of a clustering problem is a system that groups together postcode areas that tend to have a similar experience of insurance claims.

- (ii) ***Semi-supervised and reinforcement learning***

An example of semi-supervised learning is a photo app that groups photos featuring people with a similar appearance and then allows the user to name the people in order to add their names automatically to new photos

An example of reinforcement learning is a voice recognition app that adapts over time to the user's voice.

4.1 Supervised learning

Supervised learning is associated with predictive models in which the output is specified. Here the machine is given a specific aim (eg to use the variables in the data to develop a model to predict whether a person will default on a loan), and the algorithm will try to converge on the parameters of the model which provide the 'best' prediction.

Examples relevant to the actuarial profession might be:

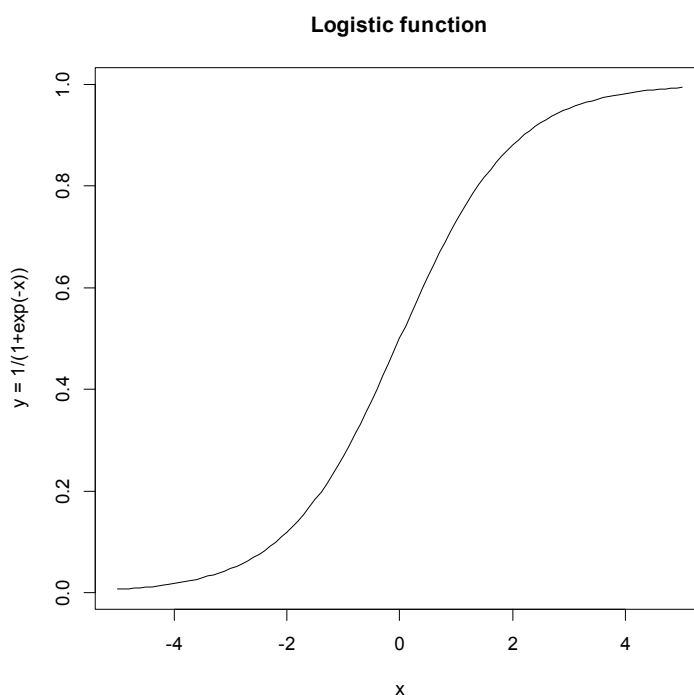
- the prediction of future lifetime at age x , T_x , or survival probabilities from age x , $P(T_x > t)$
- the prediction of the risk of claims being made on certain classes of insurance.

A distinction can be made between supervised learning that involves the prediction of a numerical value (such as future lifetime) and prediction of which category a case falls into (will a person default on a loan – yes or no?). For predicting numerical values, regression models are the normal approach, whereas predicting which category a case falls into is essentially a classification problem, and different algorithms, such as decision trees, are used. However, this distinction between regression and classification is somewhat fuzzy, as there are regression models, such as logistic regression or probit models, where the dependent variable is categorical. (These are examples of generalised linear models, which were covered in Subject CS1.)

Probit models (short for 'probability unit') produce outputs that can only take one of two values, eg Yes / No or 0 / 1.

Logistic regression is based on the logistic function $f(x) = \frac{1}{1 + e^{-x}}$, shown in the graph below.

This function converts an input value, which can be anywhere in the entire range $-\infty < x < \infty$, to an output value on a continuous scale between 0 and 1. If we interpret the output value as a probability, we can convert it to a categorical output by saying that values exceeding a specified value p (eg $p = 0.5$) correspond to Yes, while smaller values correspond to No.



This graph was plotted using the R code:

```
logistic<-function(x) 1 / (1+exp(-x))
plot(logistic,xlim=c(-5,5),main="Logistic function",
ylab="y = 1 / (1+exp(-x))")
```



Question

Give another example where classification is achieved by converting a numerical output into a categorical value.

Solution

A familiar example is where a student's numerical score on an exam, *eg* 65%, is converted into an exam grade, *eg* Pass, Fail, *etc.*

Within classification algorithms, a distinction can be made between models that generate classifications and those that discriminate between classes.

For example, the first type of model would include an algorithm that identifies people in a user's photo albums in a photo app, while the second type would include one that 'tags' a user's 'friends' in a social media app. In the first case, the algorithm groups together faces that look similar without knowing in advance the names of the people or how many there will be, whereas in the second case, it just has to match the faces to a list of other users whose names are already known.

Consider the case where we have a categorical output value y and data (covariates), $x_1, x_2, \dots$. The aim is to predict into which category of y case i will fall given the values of the covariates for case i , $x_{1i}, x_{2i}, \dots$.

For example, we might have a set of historical texts written by different authors (y) and we wish to identify the most likely author based on the frequency of certain words in the text ($x_1, x_2, \dots$).

One approach is to model the joint probabilities $P(x_1, x_2, \dots, y)$. This generates a classification scheme. It is then possible to evaluate the conditional probability of being in category y , given $x_1, x_2, \dots$ as:

$$P(y | x_1, x_2, \dots) = \frac{P(x_1, x_2, \dots, y)}{P(x_1, x_2, \dots)}$$

One problem with this approach is that the number of separate probabilities $P(x_1, x_2, \dots, y)$ to be computed increases exponentially with the number of covariates x_j .

For example, if we were considering ten words, which can each be recorded as frequent (=1) or infrequent (=0), we would already have $2^{10} = 1,024$ different combinations of the form $x_1 = 1, x_2 = 0, \dots, x_{10} = 1$.

This, however, can be overcome by assuming that, given the classes y , the covariates x_j ($j = 1, \dots, J$) are independent.

We would then only have to consider the ten different probabilities for $x_1 = 1, x_2 = 0, \dots, x_{10} = 1$ individually.

With this assumption, we have:

$$P(x_1, x_2, \dots, y) = P(y) \prod_{j=1}^J P(x_j | y)$$

This is called the *naïve Bayes classifier*.

An alternative method is to model the conditional probability $P(y | x_1, x_2, \dots)$ directly, and to find, say, a linear combination of the x_k that best discriminates between the categories of y . This is the aim of a method known as *discriminant analysis*, which is effectively the same as *logistic regression*.

Other supervised learning techniques described in machine learning textbooks include the *perceptron*, *neural networks* and *support vector machines*.

Perceptrons and *neural networks* use interconnected layers of artificial neurons that can be activated or deactivated in a way that mimics the behaviour of the neurons in animal brains.

A *support vector machine* is a classification algorithm that considers the input data values as a vector defining a point in space and tries to place hyperplanes in a way that segregates the points most effectively.

4.2 Unsupervised learning

Other machine learning techniques operate without a target for the algorithm to aim at. We might, for example, set the machine the task of identifying clusters within the data.

Given a set of covariates, the idea is that the machine should try to find groups of cases which are similar to one another but different from cases in other groups. In the language we used in the exposed to risk chapter, we try to divide the data into homogeneous classes. However, we may not tell the machine in advance what the characteristics of each of these classes should be, or even how many such classes there should be. We allow the machine to determine these given a set of rules which form part of the algorithm. Machine learning where the output is not specified in advance is called unsupervised learning.

Examples of unsupervised learning techniques include *cluster analysis*, and the use of association rules such as the *apriori algorithm*.

The *apriori algorithm* is a machine learning technique that identifies combinations of data values that frequently occur together in a data set, *eg* where users of a music website will tend to download items by the same artist or items of the same genre. It can be used by online retailers as the basis for the 'Other customers also bought ...' recommendations or for promoting bundles of items that are frequently bought together.

Apart from their use to divide data into homogeneous classes, unsupervised learning techniques are commonly used with very large data sets. Example would be market basket analysis, which uses data generated from retail transactions to identify items which are commonly purchased together, and text analysis.

For example, the predictive text feature on a mobile phone looks for combinations of words that are commonly used together so that it can auto-complete phrases such as 'Have a happy ...' with the word 'birthday'.

4.3 Semi-supervised learning

It is possible to perform machine learning analysis by using a mixture of supervised and unsupervised learning. For example, cluster analysis could be used to identify clusters. These clusters could then be labelled using a variable y , and a supervised classification algorithm such as naïve Bayes or logistic regression used to develop predictions of the class into which each case would fall.

For example, a system that aims to identify pickpockets operating in a busy shopping street might first identify people who appear several times throughout the day with the same clothing. These people's faces could then be matched against a database of known offenders.

This makes obvious sense if the clusters identified by the unsupervised learner make substantive sense for the problem at hand. But even if your clusters do not make sense to you (a human), you will have constructed a machine called an *autoencoder* – which can considerably speed up any future modelling analysis.

An *autoencoder* compresses the raw data by focusing on features that appear to be significant, eg it might identify the different types of object that appear in a photo, even though it doesn't know what they actually are.

4.4 Reinforcement learning

In *reinforcement learning* the learner is not given a target output in the same way as with *supervised* learning. The learner uses the input data to choose some output, and is then told how well it is doing, or how close the chosen output is to the desired output. The learner can then use this information as well as the input data to choose another hypothesis.

Example

Imagine a world that can be modelled as a finite-state discrete-time stochastic process with state space S . An agent in this world who is in state u at time t can take many possible actions, A_t , and each of these actions will result in a probability that the agent is in state v at time $t + 1$. We can define two functions:

- the state transition function, $P(X_{t+1} = v \mid X_t = u, A_t)$, and
- the observation, or output function $P(Y \mid X_t = u, A_t)$.

An example to have in mind here would be a fund manager who is investing the assets of a pension fund. Here state u might be the value of the assets at the time $(t, t + 1, \text{etc})$ of an annual valuation, A_t might be a particular investment strategy that the manager could follow and Y might be some measure of the fund's solvency (which will be affected by unknown factors on the liability side).

Some values of Y are more desirable than others, and we want the agent to take the actions which will lead to desirable outcomes of Y . How do we achieve this? The agent does not know the future, and cannot necessarily see how the actions taken at time t will enhance or reduce the probabilities of Y .

One possibility is to define a reward function $E(R_t | X_t = i, A_t)$, in which the reward, R_t depends on the probability that the action A_t will lead to desirable values of Y . The agent then tries to maximise its overall rewards (discounted as appropriate). Clearly, if the agent had full information about the model, we could treat this as a standard maximisation problem. But the agent does not know this: all the agent knows is the rewards it received for particular actions at specific time points up to the present. Reinforcement learning is the process by which the agent updates the probabilities of taking particular actions on the basis of past rewards received.

This process is reminiscent of the idea of the *actuarial control cycle* where adjustments are made periodically based on feedback from past experience to ensure that a particular strategy remains on track.

Another popular machine learning technique that involves reinforcement is *genetic algorithms*, which are based on the idea of selective breeding from biology. These create successive generations of possible solutions. Small random variations ('mutations') are introduced into each solution to create the next generation. The solutions are then tested and the ones that perform best are selected to continue to the next generation. After a few generations, a good solution may have been discovered amongst the surviving solutions.

5 Stages of analysis in machine learning

In this section we will look at the stages involved in applying machine learning and discuss some of the issues these raise.

Machine learning tasks can be broken down into a series of steps.

5.1 Collecting data

The data must be assembled in a form suitable for analysis using computers. Several different tools are useful for achieving this: a spreadsheet may be used, or a database such as Microsoft Access.

Data may come from a variety of sources, including sample surveys, population censuses, company administration systems, databases constructed for specific purposes (such as the Human Mortality Database, www.mortality.org).

During the last 20 to 30 years the size of datasets available for analysis by actuaries and other researchers has increased enormously. Datasets, such as those on purchasing behaviour collected by supermarkets, relate to millions of transactions.

5.2 Types of data

There are many different types of data we might need to deal with. The table below illustrates the 'traditional' types of data that have been used by actuaries and statisticians.

DATA TYPES				
NUMERICAL (ie numbers)		CATEGORICAL (ie not numbers)		
DISCRETE	CONTINUOUS	ATTRIBUTE (DICHOTOMOUS)	NOMINAL	ORDINAL
↓	↓	↓	↓	↓
Age last birthday	Exact age	Alive / Dead	Customer name	Date of birth (DD/MM/YY)
Number of children	Salary	Male / Female	Type of claim	Month (Jan, Feb, Mar, ...)
Number of claims	Claim amount	Claim / No claim	Occupation	Exam grade (A, B, C, ...)
		Pass / Fail	Marital status	Size (S, M, L, XL)
			Country	Agree/Don't know/Disagree
			Colour of car	

Attribute (or dichotomous) data refers to variables whose values consist of just two categories.

Ordinal variables take values that can be ordered in a natural way, whereas the values for *nominal* variables cannot.

Some variables can be classed in several ways, *eg* the number of claims could be treated as a continuous variable, rather than discrete, if the values were large. We've seen this idea before when we used a normal approximation to a binomial or Poisson distribution in Subject CS1.

Similarly, colour of car could be recorded as red, orange, yellow, *etc* (*ie* nominal data) or (if it came from a video image) it could be measured on the RGB scale (*ie* a vector of three discrete numerical values). It's also quite common to record attribute data as 1's and 0's (*ie* with discrete numerical values) to make it easier to count subsets and to calculate proportions and averages.

Nowadays, however, there are many other types of data that don't correspond to these familiar types. For example, a motor insurer might be provided with a memory card containing footage of an accident recorded on a vehicle's dash cam (dashboard camera). This will typically be a very large file containing a mixture of video and audio information, as well as structural information (*eg* markers for the start and end of each frame of the video), header information (*eg* the date it was captured, the serial number of the camera and the software version) and other embedded information such as time markers for the footage and satellite coordinates.

5.3 Exploring and preparing the data

Some forms of 'raw' data or complex data structures first need to be converted to one of the types in the table. For example, a Word .docx file, although actually just a long sequence of 1's and 0's, has a specific structure from which the page layouts, the fonts and the text itself would need to be extracted before we could analyse the content. Similarly, we would need to prepare a raw audio or video file before we could use the data it contains for speech recognition or face recognition.

This stage can be divided into several elements:

- **The data need to be prepared in such a way that a computer is able to access it and apply a range of algorithms. If the data are already in a spreadsheet, this may be a simple matter of importing the data into whatever computer package is being used to develop the algorithms. If the data are stored in complex file formats, it will be useful to convert the data to rectangular format, with one line per case and one column per variable. It is also important here to recognise the nature of the variables being analysed: are they nominal, ordinal or continuous?**
- **Cleaning the data, replacing missing values, and checking the data for obvious errors is an important stage of any analysis, including machine learning.**
- **Exploratory data analysis (EDA). In machine learning applications it is probably not a good idea to do extensive EDA, as the outcome might influence your choice of model and hypothesis set.**



Question

Suggest how the raw data could be converted to a usable 'rectangular' format in each of the following cases:

- identifying people in a photograph stored as an image file
- identifying vehicles from CCTV video footage.

Solution

- (a) We could create a separate record for each identified face in the picture and then record columns with the coordinates (or relative positions) of key features such as the centres of the eyes or the corners of the mouth. If the images are in colour, we could also include columns for eye colour, hair colour and complexion.
- (b) We could create a separate record for each vehicle with a group of columns for each character in the vehicle number plate to record distinctive features such as whether the character contains a vertical line and which corners contain white space. If the video footage is in colour, we could also record the colour of the vehicle, which we could use as a check.

These data processing tasks are themselves non-trivial and form an important part of the classification process. What we have described here is just one possible approach.

5.4 Feature scaling

Some machine learning techniques will only work effectively if the variables are of similar **scale**. We can see this by recalling that, in a linear regression model (which we covered in Subject CS1) the parameter, β_j , associated with covariate x_j , measures the impact on y of a one-unit change in x_j . If x_j is measured in, say, metres, the value of β_j will be 100 times larger than it would be with the same data if x_j were measured in centimetres.

In machine learning the weights w_j play the role of the β 's in the linear regression model. Consider the expression in Section 3.7:

$$L^*(w_1, w_2, \dots, w_J) + \lambda \sum_{j=1}^J w_j^2$$

The penalty imposed for model complexity is $\lambda \sum_{j=1}^J w_j^2$ which clearly depends on the weights and hence on the scale at which the features are measured.

The scaling of the variables is particularly important with nearest neighbour algorithms, which we will discuss later. The relative distances between the points will be heavily dependent on the units of measurement. To ensure that all the factors are taken into account, we would want the ranges of values for each variable to be comparable.

Descriptive statistics such as frequency distributions, measures of central tendency and of dispersion might be useful here to establish an appropriate scale for each feature, as will cross tabulations of nominal or ordinal data, or correlation coefficients between continuous variables. Pictorial representations, such as histograms and boxplots, are invaluable.

5.5 Splitting the table into the training, validation and training data sets

We discussed the ‘train-validate-test’ approach in Section 3.4, which involves splitting the data into three parts.

A typical split might be to use 60% of the data for training, 20% for validation and 20% for testing. However it depends on the problem and not on the data. A guide might be to select enough data for the validation data set and the testing data set so that the validation and testing processes can function, and to allocate the rest of the data to the training data set. In practice, this often leads to around a 60% / 20% / 20% split.

5.6 Training a model on the data

This involves choosing a suitable machine learning algorithm using a subset of the data. The algorithm will typically represent the data as a model and the model will have parameters which need to be estimated from the data. This stage is analogous to the process of fitting a model to data as described in the chapters on regression and generalised linear models in Subject CS1.

5.7 Validation and testing

The model should then be validated using the 20% of the data set aside for this purpose. This should indicate, for example, whether we are at risk of over-fitting our data. The results of the validation exercise may mean that further training is required.

Once the model has been trained on a set of data, its performance should be evaluated. How this is done may depend on the purpose of the analysis. If the aim is prediction, then one obvious approach is to test the model on a set of data different from the one used for development. If the aim is to identify hidden patterns in the data, other measures of performance may be needed.

5.8 Improving model performance

We can measure the performance of the model by testing it on the 20% of the data we have reserved for this purpose. The hope is that the performance of the final hypothesis g on the ‘test’ data set is similar to that achieved by the same hypothesis on the training data set. This amounts to stating that the difference between the in-sample error and the out-of-sample error $|E_{\text{in}}(g) - E_{\text{out}}(g)|$ will be generally small, or that:

$$P[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \varepsilon] \leq Z$$

where Z is some threshold which may depend on the precise task to hand (the greater the value at risk, the smaller Z).

This is the same type of inequality that we saw in Section 3.3. ‘Value at risk’ here just means the amount of money at stake, eg the cost of the damage that would be caused by a wrong decision. However, the idea is similar to the value at risk (VaR) from investment theory (covered in Subject CM2), where we aim to keep the probability of a large loss below a specified probability level.

If the performance of the model is not sufficient for the task at hand, it may be possible to improve its performance. Sometimes the combination of several different algorithms applied to the same data set will produce a performance which is substantially better than any individual model. In other cases, the use of more data might provide a boost to performance. However, except when considering very simple combinations of models, care should be taken not to overfit the evaluation set.

5.9 The reproducibility of research

It is important that data analysis be *reproducible*. This means that someone else can take the same data, analyse it in the same way, and obtain the same results. In order that an analysis be reproducible the following criteria are necessary:

- The data used should be fully described and available to other researchers.
- Any modifications to the data (eg recoding or transformation of variables, or computation of new variables) should be clearly described, ideally with the computer code used. In machine learning this is often called ‘features engineering’, whereby combinations of features are used to create something more meaningful.
- The selection of the algorithm and the development of the model should be described, again with computer code being made available. This should include the parameters of the model and how and why they were chosen.

There is an inherent problem with reproducing stochastic models (which are studied in Subject CM1), in that those of necessity have a random element. Of course, details of the random number generator seeds chosen, and the precise command and package used to generate any randomness, could be presented. However, since stochastic models are typically run many times to produce a distribution of results, it normally suffices that the distribution of the results is reproducible.



To ensure reproducibility in stochastic models in R, use the same numerical seed in the function `set.seed()`.

6 Applications: supervised learning

In this section we describe some commonly-used supervised machine learning techniques. Some of these methods are direct extensions of methods covered elsewhere in the syllabus, notably the linear regression and generalised linear models in Subject CS1 and the proportional hazards models in [Chapter 8](#) of this course.

6.1 Penalised generalised linear models

Suppose we have a set of data with J covariates $(x_1, \dots, x_J)$ and N cases (the sample size). To fit a generalised linear model, we normally specify the link function and the parameters of the model, $\beta_0, \dots, \beta_J$ (where β_0 is the intercept) and maximise the likelihood $L(\beta_0, \dots, \beta_J \mid x_1, \dots, x_J)$. However, this might not work well in certain situations.

For example, if there are many covariates, including all of them might make the model unstable (the estimators of the parameters $\beta_1, \dots, \beta_J$ will have large variances) because of correlations among the $x_1, \dots, x_J$. If we wish to use the model for prediction on new data, this is very undesirable. We want to be able to trust that the estimated values of the parameters linking the covariates to the outcome variable are solidly grounded and unlikely to shift greatly when the model is applied to a new data set. Another way of saying this is that we only want to include features which really do have a general effect on output.



Question

Give an example of this problem of correlations between covariates in an actuarial setting.

Solution

Suppose that two of the variables that a motor insurer uses to determine premiums for its policyholders are 'postcode' and 'type of car', and that the FX99 postcode area and Nano cars are usually considered to be low-risk and therefore cheap to insure. However, last year there was a single policyholder in the FX99 area who drove a Nano car and this policyholder was involved in a freak accident that led to a claim that cost £1 million.

Next year we would expect the insurer's pricing model to increase premiums for any policyholder living in the FX99 area or driving a Nano car to reflect the past experience of these two categories. Any other policyholders who happened to live in the FX99 area *and* drive a Nano car would have their premiums increased on both counts.

If the insurer also uses the different *combinations* of postcode and type of car in its pricing model (*ie* separate premiums are determined for each pairing of postcode and type of car), it would now charge a prohibitively high premium to any other policyholder who happened to live in the FX99 area *and* drive a Nano car. This is purely because the values of their covariates match those of the policyholder who made the large claim and the pricing model has put them in the same category as this policyholder and has 'tarred them with the same brush'.

So it is important for us to avoid over-parameterising our model.

One way to solve this is to choose a subset of the J covariates to include in the model. But how do we choose this? We could look at all possible subsets of J and use criteria such as the Akaike Information Criterion or the Bayesian Information Criterion.

These both exact a penalty for additional parameters. If the number of parameters is J and the sample size in the data is N :

$$\text{AIC} = \text{deviance} + 2J$$

$$\text{BIC} = \text{deviance} + [\log_e(N)]J$$

The deviance here is a measure of the average error of the model's outputs and measures the goodness of fit. The extra terms reflect the number of parameters in the model. If we aim to minimise the AIC or the BIC, this will allow us to find a good trade-off between the two objectives of obtaining a good fit to the data and minimising the number of parameters in the model.

However as J increases the number of possible subsets rises. In many machine learning applications, J is large, and the number of cases is also large, so that comparing all possible subsets is computationally infeasible.

Penalised regression involves exacting a penalty for unrealistic or extreme values of the parameters, or just having too many parameters. The penalty may be written $\lambda P(\beta_1, \dots, \beta_J)$, so that we maximise:

$$\log_e L(\beta_0, \dots, \beta_J \mid x_1, \dots, x_J) - \lambda P(\beta_1, \dots, \beta_J)$$

By maximising this expression we are aiming for a trade-off where we try to maximise the log-likelihood but, at the same time, try to minimise the penalty applied (since this is subtracted).

Two common examples of penalties are:

- ridge regression, where $P(\beta_1, \dots, \beta_J) = \sum_{j=1}^J \beta_j^2$,
- the LASSO (Least Absolute Shrinkage and Selection Operator), where $P(\beta_1, \dots, \beta_J) = \sum_{j=1}^J |\beta_j|$.

The names are based on the geometrical interpretations of these measures, which you are not expected to know.

These penalty functions assume that the distribution has been parameterised in such a way that we would expect the true values of the parameters $\beta_1, \dots, \beta_J$ to be close to zero. If this is not the case, we would subtract the target value from each β in the penalty functions shown above.

The parameter λ is called the regularisation parameter, and its choice is important. Too small a value of λ leads to over-fitting the data and the problems associated with using just the likelihood. Too large a value of λ means only gross effects of the covariates will be included in the final model, and we may miss many important effects of the covariates on the outcome.

The parameter λ is an example of a hyper-parameter.



Question

A random sample $x_1, \dots, x_n$ of n values has been taken from a Poisson distribution with unknown mean μ . The value of μ is to be estimated using the penalty function $\lambda(\mu - 5)^2$.

- (i) Explain why this particular penalty function might have been chosen.
- (ii) Write down an expression for the penalised log-likelihood function.
- (iii) Show that $\hat{\mu}$, the estimated value of μ , satisfies the equation:

$$n(\hat{\mu} - \bar{x}) + 2\lambda\hat{\mu}(\hat{\mu} - 5) = 0$$

- (iv) Calculate the value of $\hat{\mu}$ based on the sample of values 5.7, 5.4, 4.6, 5.0 and 4.9 when $\lambda = 0.2$.
- (v) Use the equation in part (iii) to show algebraically what happens to the value of $\hat{\mu}$:
 - (a) when λ is equal to zero and
 - (b) when λ is very large.
- (vi) Comment on your answers in part (v).

Solution

In this question we have just one parameter (ie $J = 1$), which is called μ (corresponding to β_1).

- (i) We might choose this penalty function if we believe the true value of μ is close to 5, as the penalty when $\mu = 5$ would be zero.
- (ii) The likelihood function for the sample is:

$$L = \prod_{i=1}^n \frac{e^{-\mu} \mu^{x_i}}{x_i!} = e^{-n\mu} \mu^{\sum x_i} \times \text{constant} = e^{-n\mu} \mu^{n\bar{x}} \times \text{constant}$$

So the log-likelihood is:

$$\log L = -n\mu + n\bar{x} \log \mu + \text{constant}$$

and the penalised log-likelihood is:

$$(\log L)^* = -n\mu + n\bar{x} \log \mu + \text{constant} - \lambda(\mu - 5)^2$$

- (iii) To maximise this, we equate the derivative with respect to the parameter μ to zero:

$$\frac{\partial(\log L)^*}{\partial \mu} = -n + \frac{n\bar{x}}{\mu} - 2\lambda(\mu - 5) = 0$$

Multiplying by μ and rearranging gives:

$$n(\mu - \bar{x}) + 2\lambda\mu(\mu - 5) = 0$$

So $\hat{\mu}$ satisfies the equation:

$$n(\hat{\mu} - \bar{x}) + 2\lambda\hat{\mu}(\hat{\mu} - 5) = 0$$

(iv) For this sample, we have:

$$n = 5 \quad \text{and} \quad \bar{x} = \frac{1}{5}(5.7 + 5.4 + 4.6 + 5.0 + 4.9) = \frac{25.6}{5} = 5.12$$

So with $\lambda = 0.2$, the equation in part (iii) becomes:

$$5(\hat{\mu} - 5.12) + 2(0.2)\hat{\mu}(\hat{\mu} - 5) = 0$$

This can be rearranged to give:

$$0.4\hat{\mu}^2 + 3\hat{\mu} - 25.6 = 0$$

Using the quadratic formula:

$$\hat{\mu} = \frac{-3 \pm \sqrt{3^2 - 4(0.4)(-25.6)}}{2(0.4)} = \frac{-3 \pm \sqrt{49.96}}{0.8} = -12.585 \quad \text{or} \quad 5.085$$

Since μ must be positive, the required estimate is $\hat{\mu} = 5.085$.

(v)(a) If $\lambda = 0$, the equation in part (iii) becomes:

$$n(\hat{\mu} - \bar{x}) = 0 \Rightarrow \hat{\mu} = \bar{x}$$

(v)(b) If we divide the equation in part (iii) by λ , we get:

$$\frac{n(\hat{\mu} - \bar{x})}{\lambda} + 2\hat{\mu}(\hat{\mu} - 5) = 0$$

As $\lambda \rightarrow \infty$, this becomes:

$$2\hat{\mu}(\hat{\mu} - 5) = 0 \Rightarrow \hat{\mu} = 0 \quad \text{or} \quad \hat{\mu} = 5$$

Since the value of μ must be strictly positive, the required estimate would be $\hat{\mu} = 5$.

(vi) If we apply no penalty, as in part (v)(a), the method reduces to maximum likelihood estimation and we get the usual estimate for μ , ie the sample mean of 5.12.

If we apply a high penalty to values that are not close to the target value of 5, as in part (v)(b), the method will produce a value close to 5, irrespective of the actual values in the sample.

As expected, the estimated value of 5.085 lies between these two values.

6.2 Naïve Bayes classification

Recall from Subject CS1 that if $B_1, B_2, \dots, B_R$ constitute a partition of a sample space S and $P(B_i) \neq 0$ for $i = 1, 2, \dots, R$, then for any event A in S such that $P(A) \neq 0$:

$$P(B_r | A) = \frac{P(A|B_r)P(B_r)}{P(A)} \text{ for } r = 1, 2, \dots, R$$

where:

$$P(A) = \sum_{i=1}^R P(A|B_i)P(B_i)$$

This is *Bayes' Theorem*, which allows us to 'invert' conditional probabilities, i.e. to work out the values of the probabilities $P(B_r | A)$ when we know the probabilities $P(A | B_r)$.



Question

Derive Bayes' Theorem.

Solution

The proof uses the definition of conditional probabilities, $P(X | Y) = \frac{P(X, Y)}{P(Y)}$, and the equivalent identity $P(X, Y) = P(X | Y)P(Y)$.

Using the definition of conditional probabilities, the probability we want to find is:

$$P(B_r | A) = \frac{P(B_r, A)}{P(A)}$$

Using the identity above, the numerator on the right-hand side can be written as:

$$P(B_r, A) = P(A | B_r)P(B_r)$$

If we condition the denominator on the different possible values of B_r , we can write it as:

$$P(A) = P(A | B_1)P(B_1) + P(A | B_2)P(B_2) + \dots + P(A | B_R)P(B_R) = \sum_{r=1}^R P(A | B_r)P(B_r)$$

If we substitute these in, we then get Bayes' formula:

$$P(B_r | A) = \frac{P(A | B_r)P(B_r)}{P(A)} = \frac{P(A | B_r)P(B_r)}{\sum_{r=1}^R P(A | B_r)P(B_r)}$$

Naïve Bayes classification uses this formula to classify cases into mutually exclusive categories on some outcome y , on the basis of a set of covariates $x_1, \dots, x_J$. The events A are equivalent to the covariates taking some set of values, and the partition $B_1, B_2, \dots, B_R$ is the set of values that the outcome can take.

Here B_1 corresponds to the *vector* of covariates $(x_{11}, x_{21}, \dots, x_{J1})$ for the first individual.

Suppose the outcome is whether or not a person will survive for 10 years. Let $y_i = 1$ denote the outcome that person i survives, and $y_i = 0$ denote the outcome that person i dies. Then, if we have J covariates, we can write:

$$P(y_i = 1 | x_{1i}, \dots, x_{Ji}) = \frac{P(x_{1i}, \dots, x_{Ji} | y_i = 1)P(y_i = 1)}{P(x_{1i}, \dots, x_{Ji})}$$

This is difficult to estimate because all possible combinations of the $x_1, \dots, x_J$ need to be estimated, and all combinations are unlikely to be in your data set.

This is a particular problem here because in practice many of the combinations of values will not be present in the sample, so the corresponding probabilities cannot be estimated. For example, for a motor insurer that uses several rating factors (eg age of policyholder, occupation, make of car, age of car, postcode area), many of the subsets will be empty.

The naïve Bayes algorithm assumes that the values of the x_i are independent, conditional on the value of y_i .

So we are assuming that:

$$P(x_{11}, x_{21}, \dots, x_{J1} | y_i = 1) = P(x_{11} | y_i = 1) \times P(x_{21} | y_i = 1) \times \dots \times P(x_{J1} | y_i = 1)$$



Question

Illustrate why this assumption might not be accurate for a motor insurer using the rating factors age of policyholder, occupation, make of car, age of car, postcode area.

Solution

As an example, 25% of policyholders may be under 25, 20% may drive high performance cars and 40% may drive cars less than 3 years old.

However, it is unlikely that 2% (ie $25\% \times 20\% \times 40\%$) of policyholders would be under 25 with high performance cars under 3 years old, as young drivers are unlikely to be able to afford such vehicles – or to be able to pay to insure them.

This allows the formula to be re-written:

$$P(y_i = 1 | x_{1i}, \dots, x_{Ji}) = \frac{P(x_{1i} | y_i = 1)P(x_{2i} | y_i = 1) \dots P(x_{Ji} | y_i = 1) P(y_i = 1)}{P(x_{1i}, \dots, x_{Ji})}$$

so that:

$$P(y_i = 1 | x_{1i}, \dots, x_{Ji}) \propto P(y_i = 1) \prod_{j=1}^J P(x_{ji} | y_i = 1)$$

For a given set of values of $x_{1i}, \dots, x_{Ji}$, the denominator $P(x_{1i}, \dots, x_{Ji})$ doesn't change. So we can treat this as a constant in the calculations and just look at the relative values – hence the proportional sign. To find the actual values of the individual probabilities, we can just divide by the total, to produce a set of probabilities that add up to 1.



Question

A motor insurer has analysed a sample of 1,000 claims for three different geographical regions split by the size of claim (Small / Medium / Large). It has then classified them according to whether they proved to be fraudulent or genuine claims. The results are shown in the tables below:

FRAUDULENT	Region 1	Region 2	Region 3	Total
Small	3	0	6	9
Medium	10	5	20	35
Large	3	1	2	6
Total	16	6	28	50

GENUINE	Region 1	Region 2	Region 3	Total
Small	57	38	70	165
Medium	250	95	180	525
Large	176	46	38	260
Total	483	179	288	950

- (i) Give a formula that could be used to estimate the probability that a new claim from Region 3 for a Medium amount will prove to be fraudulent.
- (ii) Estimate the probability that each of the following types of new claim will be fraudulent:
 - (a) a claim from Region 3 for a Medium amount
 - (b) a claim from Region 1 for a Large amount
 - (c) a claim from Region 2 for a Small amount.

Solution

(i) Using Bayes' Theorem (and obvious abbreviations for the events), we have:

$$P(F | R3, M) = \frac{P(F, R3, M)}{P(R3, M)} = \frac{P(R3, M | F)P(F)}{P(R3, M | F)P(F) + P(R3, M | G)P(G)}$$

(ii)(a) From the tables above, we have:

$$P(R3, M | F) = \frac{20}{50} = 0.4, \quad P(R3, M | G) = \frac{180}{950},$$

$$P(F) = \frac{50}{50 + 950} = 0.05, \quad P(G) = \frac{950}{50 + 950} = 0.95$$

$$\text{So: } P(F | R3, M) = \frac{0.4 \times 0.05}{0.4 \times 0.05 + 180/950 \times 0.95} = 0.1$$

So the estimated probability that a claim from Region 3 for a Medium amount is fraudulent is 10%.

In fact, we can do this calculation directly from the table without introducing probabilities. For Region 3 and Medium amounts there were 20 fraudulent claims and 180 genuine claims. So the estimated probability that a claim from Region 3 for a

Medium amount is fraudulent is $\frac{20}{20 + 180} = 0.1$, ie 10%.

(ii)(b) Similarly, the estimated probability that a claim from Region 1 for a Large amount is fraudulent is $\frac{3}{3 + 176} = 0.168$, ie 1.68%.

(ii)(c) Since there were no fraudulent claims from Region 2 for a Small amount in the sample, the estimated probability that a claim from Region 2 for a Small amount is fraudulent is 0.

6.3 Decision trees (classification and regression tree algorithm)

Classification and regression trees (CART) is a term introduced by Leo Breiman to refer to decision tree algorithms that can be used for classification or regression in predictive modelling problems.

Classically, this algorithm is referred to as 'decision trees' but on some platforms like R they are referred to by the more modern term CART.

The CART algorithm provides a foundation for important algorithms like:

- **bagged decision trees**
- **random forest**
- **boosted decision trees.**

We will not discuss these three types of decision trees in detail in this course, but we will provide a very brief insight into these approaches.

In *bagged decision trees*, we create random sub-samples of our data with replacement, train a CART model on each sample, and (given new data) calculate the average prediction from each model.

Random forests apply a method based on averaging a number of randomly generated decision trees.

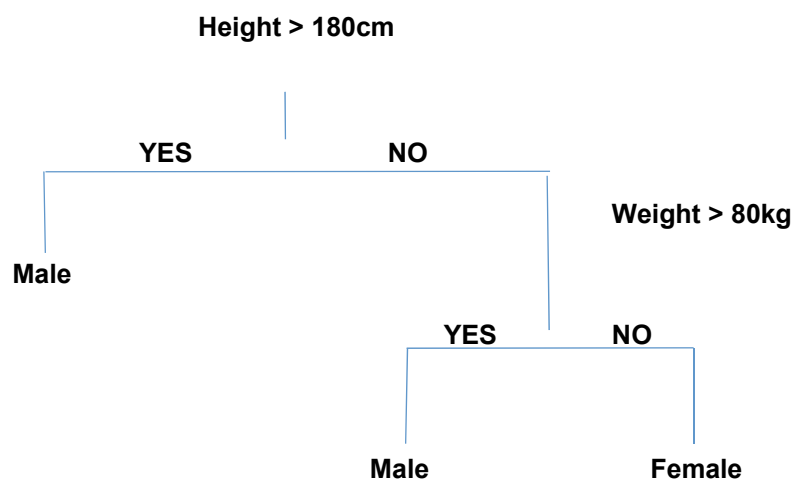
Boosting here refers to a method of repeatedly making small adjustments to improve the effectiveness of a model by reducing the residual error.

The representation for the CART model is a binary tree.

Each root node on a tree represents a single input variable x and a split point on that variable (assuming the variable is numeric).

The leaf nodes of the tree contain an output variable y which is used to make a prediction.

Given a dataset with two inputs of height in centimetres and weight in kilograms, the output of gender as male or female, below, is a crude example of a binary decision tree.



Example

Given an input of [height = 60cm, weight = 65kg] the above tree would be traversed as follows:

Node 1: Height > 180cm? No

Node 2: Weight > 80kg? No

Therefore, my result is: Female



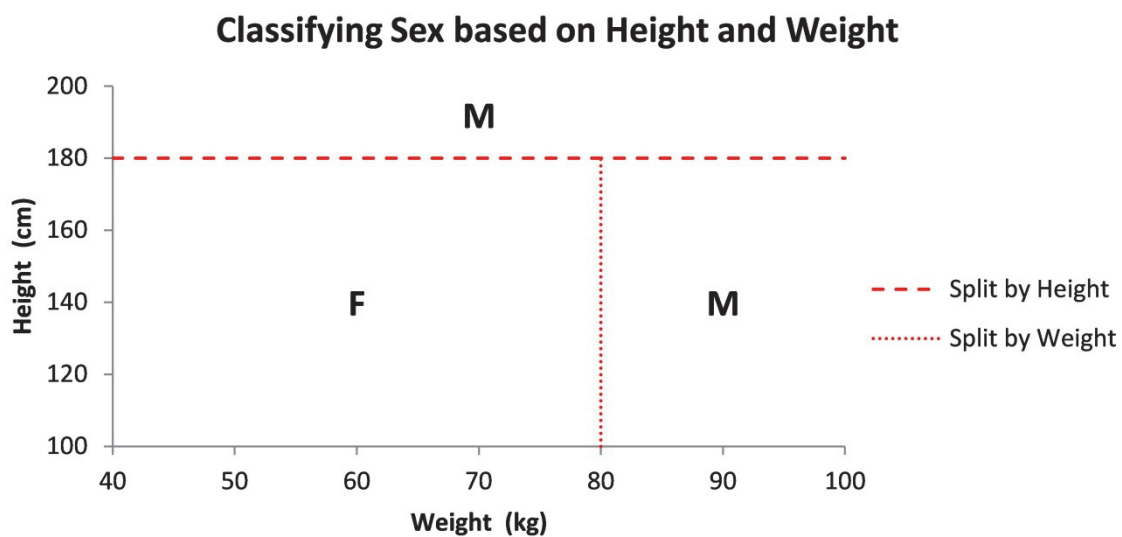
Question

- (i) Comment on the usefulness of this tree when applied to an ActEd tutor whose height is 175cm and whose weight is approximately 80kg.
- (ii) Explain the rationale behind it.

Solution

- (i) Using this tree, this tutor would be on the cusp between Male and Female. The prediction of the tutor's gender would not be reliable, as it would depend on whether their weight was above or below 80kg on a particular day.
- (ii) The rationale is that tall people (Height > 180cm) tend to be male, so we can separate them out at the first stage. Of the remaining people, males tend to be heavier than females, so we can split these at a level that is likely to a reasonable boundary between males and females (80kg, say).

We can represent this tree using the rectangle below.



With the binary tree representation of the CART model described above, making predictions is relatively straightforward.

Given a new input, the tree is traversed by evaluating the specific input at the root node of the tree.

A learned binary tree is a partitioning of the input space. You can think of each input variable as a dimension on a p -dimension space. The decision tree splits this up into rectangles (when $p = 2$ input variables) or hyper-rectangles with more inputs.

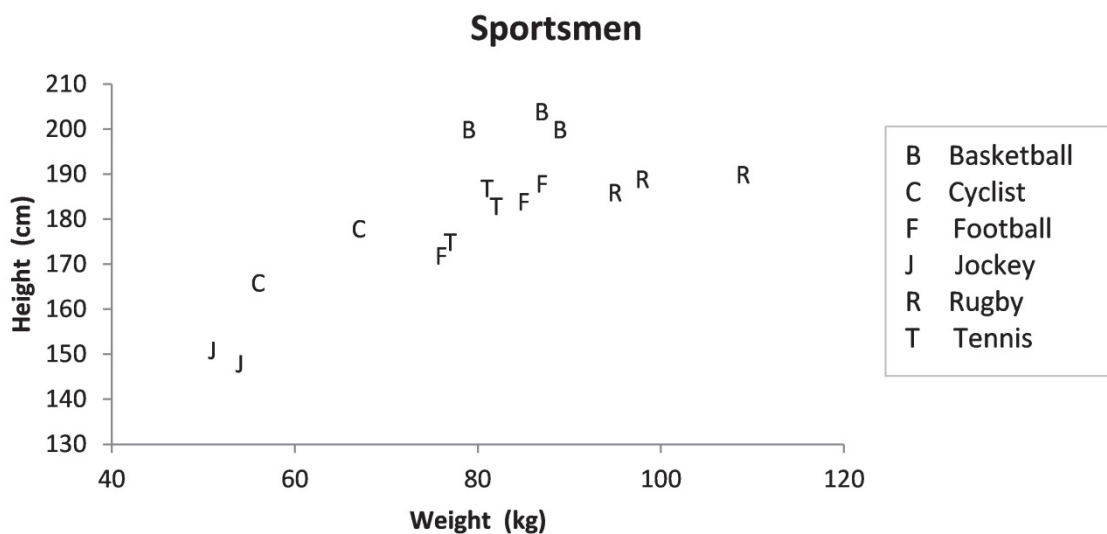
In the model above we have $p = 2$ dimensions, *ie* there are two input variables, Height and Weight.

New data is filtered through the tree and lands in one of the rectangles and the output value for that rectangle is the prediction made by the model. This gives an intuition for the type of decisions that a CART model can make, eg boxy decision boundaries.



Question

The graph below shows the heights and weights of a sample of 16 sportsmen, which were measured at the start of a competition. Devise a binary decision tree that could be used to identify the most likely sport for other sportsmen who are competing, based only on their height and weight. Explain the rationale behind your algorithm.



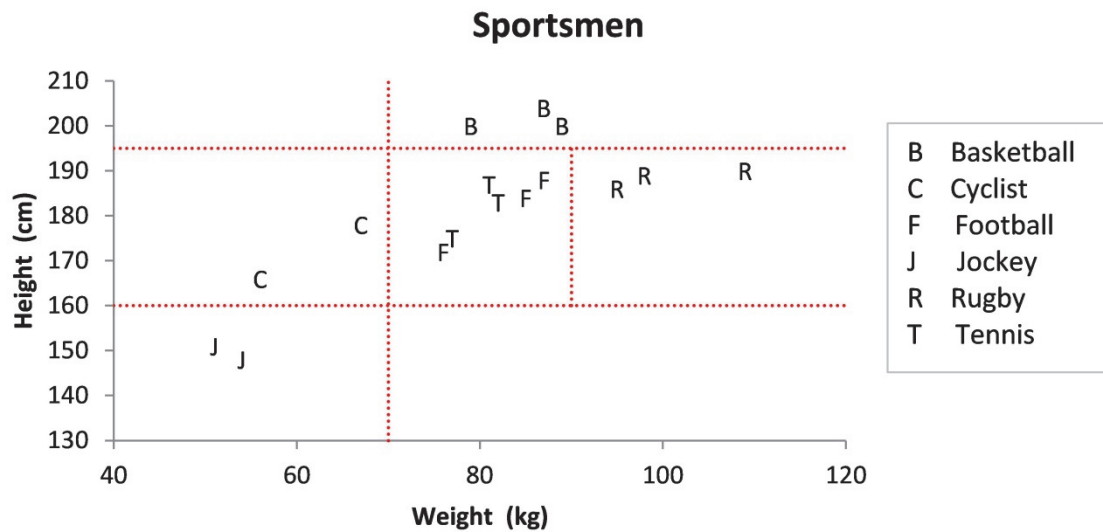
Solution

We can easily section off the basketball players, who tend to be tall, and the jockeys, who tend to be short.

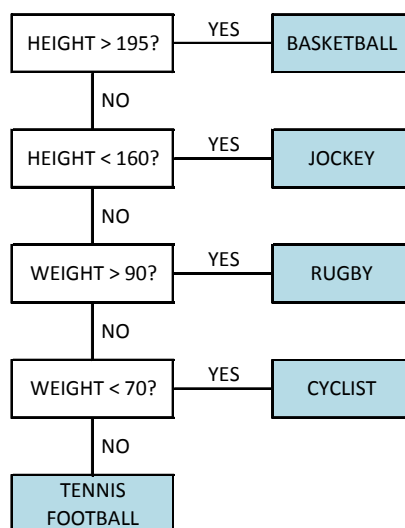
We can then section off the rugby players, who tend to be heavier, and the cyclists, who tend to be lighter.

However, the tennis players and footballers are intermixed, so we can't distinguish effectively between these two sports.

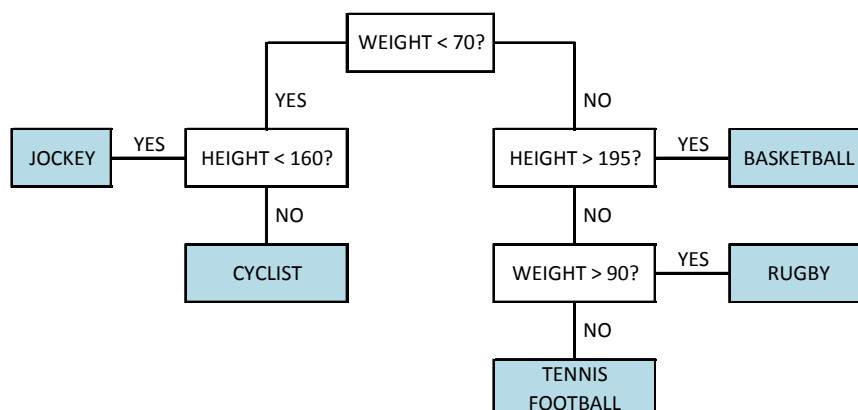
This leads to the partitions shown in the graph below.



Here is one possible algorithm based on these partitions.



This is not the only way to achieve the same result. For example, we could instead start with the test 'Weight < 70?' This alternative version has the slight advantage that the maximum number of tests that might be required is now 3, rather than 4.



Greedy splitting

Creating a binary decision tree is a process of dividing up the input space. A ‘greedy’ approach is used to divide the space, called recursive binary splitting.

This is a numerical procedure where all the values are lined up and different split points are tried and tested using a cost function. The split with the best cost (lowest cost because we minimize cost) is selected.

All input variables and all possible split points are evaluated and chosen in a greedy manner (ie the very best split point is chosen each time).

This is the approach we used in the previous example for sportsmen. In this particular example, most of the split points were clear-cut. However, to separate the tennis players and footballers, we would need to try different possible splits and then choose the one that minimises the number of incorrect classifications based on the training sample. So we would be applying a zero-one cost function where the cost is 1 unit if the answer is wrong and 0 if it is right.

The word ‘greedy’ here means that, at each stage, we just choose the split that appears to be the most effective at separating the remaining elements, without thinking ahead to the consequences this might have on the later splits. So we just ‘bite off as much as we can’ at each stage.

With just two variables, it is easy to visualise the best splits on a 2D graph. However, in higher dimensions, this is not so easy and we will need to calculate the effectiveness of different possible splits.

For regression problems, the cost function that is minimized to choose split points is the sum squared error across all training samples that fall within the rectangle:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where y_i is the output for the training sample and $\hat{y}_i$ is the predicted output for the rectangle.

For classification, the Gini index function is used, which provides an indication of how ‘pure’ the leaf nodes are (ie how mixed the training data assigned to each node is):

$$G = \sum_k p_k (1 - p_k)$$

This formula gives the Gini index for one of the final nodes in the tree or for a rectangle in the diagram.

Here p_k is the proportion of training instances with class k in the rectangle of interest. A node that has all classes of the same type (perfect class purity) will have $G = 0$, whereas a node that has a 50-50 split of classes for a binary classification problem (worst purity) will have $G = 0.5$.

The Gini index is a measure of inequality of a distribution that was introduced by the Italian statistician Corrado Gini. It was originally used to measure inequality in the distribution of income levels within a population.

It is calculated as the probability that, if two items are selected at random (with replacement), they will be of different types (ie if the first item is of type k , the second item will not be of type k). So, if all the items are the same, the probability will be 0, whereas, if they are all different, it will be close to 1.

For a binary classification problem, this can be re-written as:

$$G = 2p_1p_2$$

or
$$G = 1 - (p_1^2 + p_2^2)$$

This follows since $p_1 + p_2 = 1$, so that $(p_1 + p_2)^2 = p_1^2 + p_2^2 + 2p_1p_2 = 1$.

The Gini index calculation for each node is weighted by the total number of instances in the parent node. The Gini score for a chosen split point in a binary classification problem is therefore calculated as follows:

$$G = \left[1 - (g_{1,1}^2 + g_{1,2}^2)\right] \times \frac{n_{g_1}}{n} + \left[1 - (g_{2,1}^2 + g_{2,2}^2)\right] \times \frac{n_{g_2}}{n}$$

This formula gives the Gini index for the split point, ie it is a combined value that covers both of the nodes that meet at that point.

Here:

- $g_{1,1}$ is the proportion of instances in group 1 for class 1, $g_{1,2}$ for group1 and class 2
- $g_{2,1}$ is the proportion of instances in group 2 for class 1, $g_{2,2}$ for group 2 and class 2
- n_{g_1} and n_{g_2} are the total number of instances in groups 1 and 2
- n is the total number of instances we are trying to group from the parent node.

We can write the general formula for the Gini index for the whole tree as:

$$G = \sum_{nodes} \frac{n_{node}}{n} \left(1 - \sum_{k=1}^m p_k^2\right)$$

where the sum is taken over all the nodes and n_{node} is the number of items at the node we are currently evaluating.



Question

Calculate the Gini index for each of the final nodes in the decision tree for the sportsmen and hence calculate the overall Gini index for the tree. Comment on your answer.

Solution

In this example the classes are the different sports.

We need to consider each of the shaded nodes (which are the same for both of the trees given).

For the node containing 3 tennis players and 3 footballers we have $p_1 = p_2 = \frac{3}{6} = \frac{1}{2}$.

$$\text{So: } G = 1 - (p_1^2 + p_2^2) = 1 - \left[\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 \right] = \frac{1}{2}$$

Each of the other nodes is 'pure', with $p_1 = 1$. So these nodes have a Gini index of 0.

The overall Gini index for the tree is obtained by weighting these values by the number of data points:

$$G = \frac{1}{2} \times \frac{6}{16} + 0 \times \frac{10}{16} = \frac{3}{16} = 0.1875$$

This value is quite low, indicating that the tree is reasonably effective at classifying the sportsmen.

Stopping criterion

The recursive binary splitting procedure described above needs to know when to stop splitting as it works its way down the tree with the training data.

The most common stopping procedure is to use a minimum count on the number of training instances (ie the data items in our sample) assigned to each leaf node. If the count is less than some minimum then the split is not accepted and the node is taken as a final leaf node.

So, for example, we could say that if we try to split a node and it would result in fewer than, say, 5 data items in our sample on one of the new nodes, then we stop and leave the node as it is.

The minimum number we choose here (ie 5) is another example of a hyper-parameter.

The count of training members (ie the data items in our sample) is tuned to the dataset, eg 5 or 10. It defines how specific to the training data the tree will be. Too specific (eg a count of 1) and the tree will overfit the training data and likely have poor performance on the test set.

If we keep splitting until there is just one item on each of the final leaves in the tree, we will end up adding decision tests that are very specific to the particular data in the training set. For example, if there happened to be a female in the training data whose height was 183cm, we might include a test 'Height = 183cm?' to allocate her correctly. This would give a perfect result for this particular training set, but would not work for other data sets. So this would be an example of overfitting.

Pruning the tree

The stopping criterion is important as it strongly influences the performance of the tree. Pruning may be used after learning to further enhance the tree's performance.

The complexity of a decision tree is defined as the number of splits in the tree. Simpler trees are preferred. They are easy to understand (you can print them out and show them to subject matter experts), and they are less likely to overfit your data.

The fastest and simplest pruning method is to work through each leaf node in the tree and evaluate the effect of removing it using a hold-out test set. Leaf nodes are removed only if it results in a drop in the overall cost function on the entire test set. You stop removing nodes when no further improvements can be made.

The 'hold-out test set' just refers to another data set that wasn't used in training the model.

More sophisticated pruning methods can be used such as cost complexity pruning (also called 'weakest link pruning') where a learning parameter (alpha) is used to weigh whether nodes can be removed based on the size of the sub-tree.

7 Applications: unsupervised learning

7.1 K-means clustering

Suppose we have a set of data consisting of several variables (or features) measured for a group of individuals. These might relate to demographic characteristics, such as age, occupation, gender. Alternatively, they might relate to life insurance policies for which we have information such as sales channel, policy size, postcode, level of underwriting, etc.

We might ask whether we can identify groups (clusters) of policies which have similar characteristics. We may not know in advance what these clusters are likely to be, or even how many there are in our data.

Here we would be particularly interested in how we could group policies by occupation or postcode, as it would not be obvious at the outset how to do this in a logical way so that ones with similar outcomes are grouped together.

There are a range of clustering algorithms available, but many are based on the *K-means algorithm*. This is an iterative algorithm which starts with an initial division of the data into *K* clusters, and adjusts that division in a series of steps designed to increase the homogeneity within each cluster and to increase the heterogeneity between clusters.

The *K-means* algorithm proceeds as follows. Let us suppose we have data on *J* variables.

1. Choose a number of clusters, *K*, into which the data are to be divided. This could be done on the basis of prior knowledge of the problem. Alternatively, the algorithm could be run several times with different numbers of clusters to see which produces the most satisfactory and interpretable result. There are various measures of within- and between-group heterogeneity, often based on within-groups sums of squares. Comparing within-groups sums of squares for different numbers of clusters might identify a value of *K* beyond which no great increase in within-group homogeneity is obtained.

These measures are based on the same identity for partitioning the sum of squares into 'between groups' and 'within groups' that forms the basis for the technique of analysis of variance (ANOVA), which is mentioned in the regression chapter in Subject CS1:

$$\underbrace{\sum \sum (y_{ij} - \bar{y}_{..})^2}_{\text{Total sum of squares}} = \underbrace{\sum \sum (\bar{y}_{i.} - \bar{y}_{..})^2}_{\text{Between groups}} + \underbrace{\sum \sum (y_{ij} - \bar{y}_{i.})^2}_{\text{Within groups}}$$

2. Identify (perhaps arbitrarily) cluster centres in the *J*-dimensional space occupied by the data. This initial location of the centres could be done on the basis of prior knowledge of the problem to hand, or by random assignment of cases.
3. Assign cases to the cluster centre which is nearest to them, using some measure of distance. One common measure is Euclidean distance:

$$\text{dist}(x, k) = \sqrt{\sum_{j=1}^J (x_j - k_j)^2}$$

Here x_j is the standardised value of covariate j for case x , and k_j is the value of covariate j at the centre of cluster k ($k = 1, \dots, K$). Note that it is often necessary to standardise the data before calculating any distance measure, for example by assuming a normal distribution using z-scores or by assuming a uniform distribution on (x_a, x_b) , where x_a and x_b are the lowest and highest observed values of covariate x .

This is because the measure of distance is based purely on the numerical values of the variables. If some of the variables take large values because of the units of measurement that have been adopted, these variables will dominate the calculations and the other variables will effectively be ignored.

4. **Calculate the centroid of each cluster, using the mean values of the data points assigned to that cluster. This centroid becomes the new centre of each cluster.**

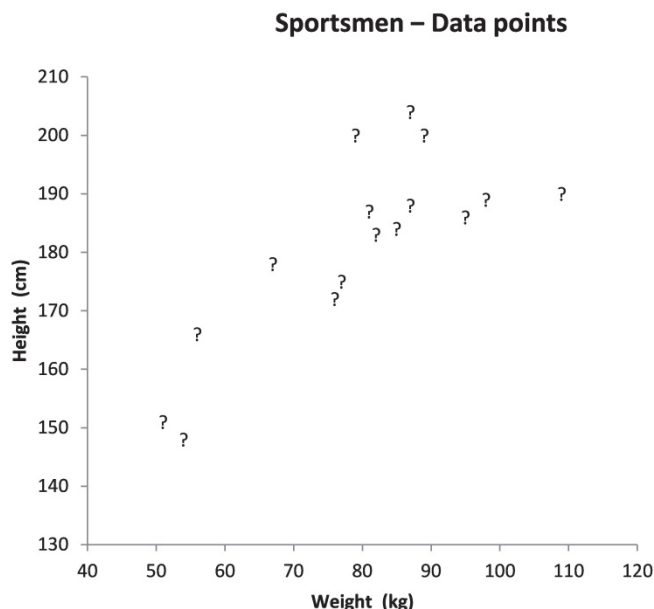
The centroid is just the centre of gravity of the data points when each has the same weight. To calculate it, we find the average of each 'coordinate' in the data set.

5. **Re-assign cases to the nearest cluster centre using the new cluster centres.**

Iterate steps 4 and 5 until no re-assignment of cases takes place.

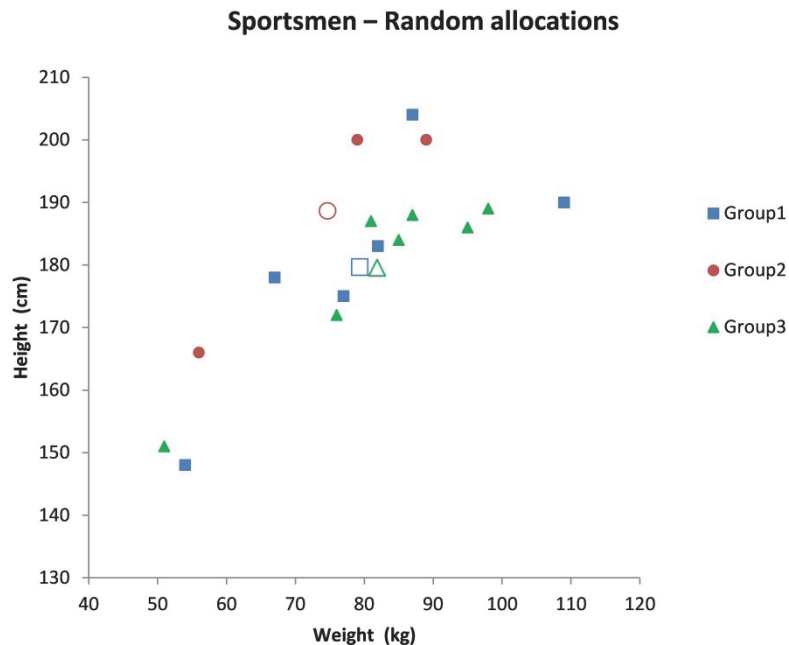
Example

The graph below shows the heights and weights of the same sample of 16 sportsmen from the previous example, but without showing the sports they were competing in. We have rescaled the vertical axis so that the spacing is the same on each axis. This ensures that the distances between the points appear correctly.



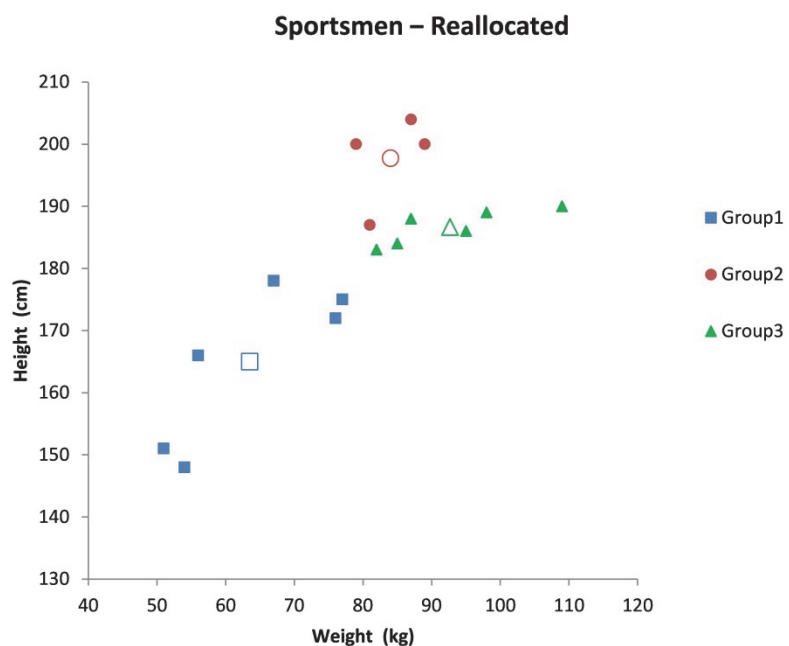
We can use the k -means method with $k = 3$ to try to identify any natural clusters in the data.

We start by allocating each of the data points to one of the 3 clusters at random. We've used squares, circles and triangles to distinguish the three groups. We then need to calculate the position of the centroid of each cluster by averaging the x and y coordinates of the data points within each cluster. The centroids are indicated by the larger hollow shapes.

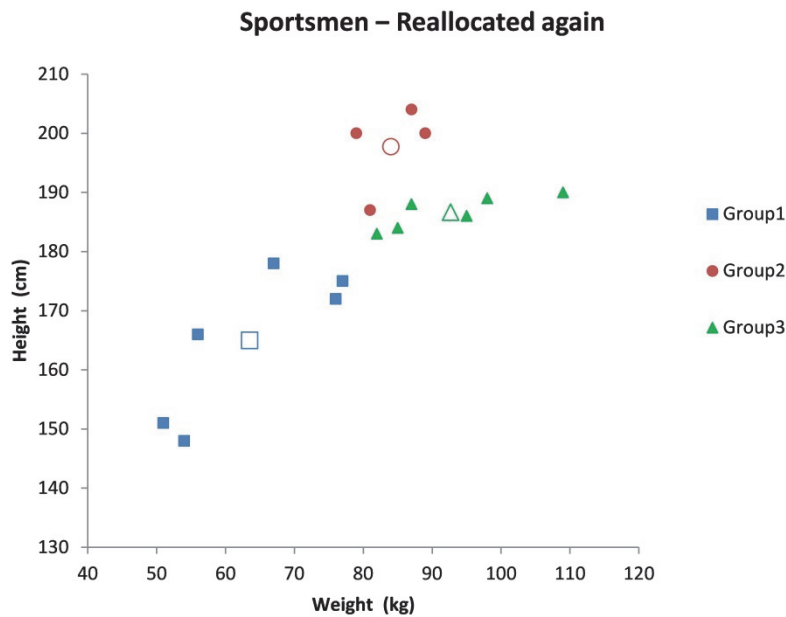


Since the initial allocation was random, the three centroids are all quite close together at this stage.

We now reallocate each data point to the cluster whose centroid it is nearest to and then recalculate the positions of the centroids of the new groups.



Again, we reallocate each data point to the cluster whose centre it is nearest to.



We can see that there has been no change to the allocations from the previous diagram. So the algorithm has now converged, with the points allocated to the three groups indicated by the different shapes.

Comment

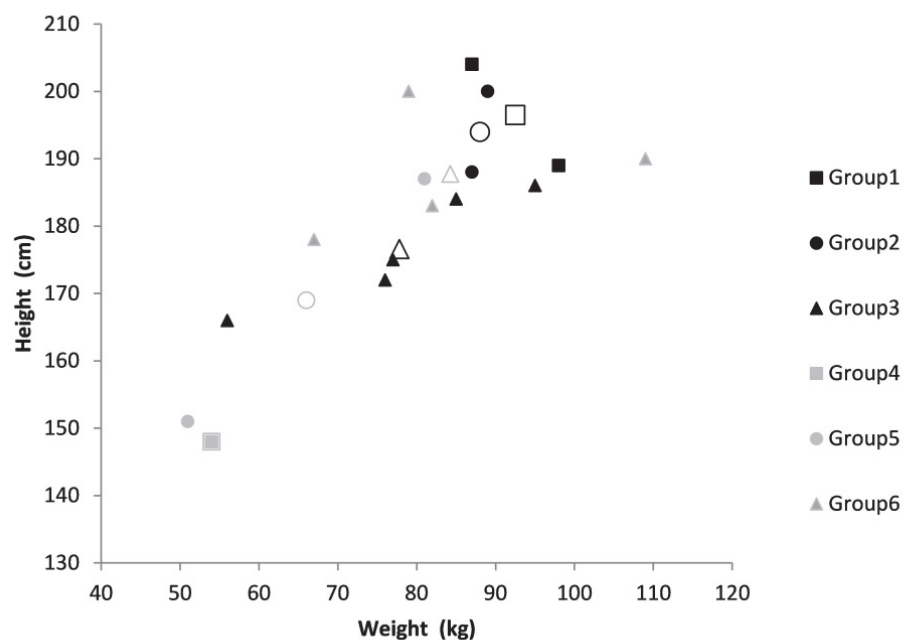
If we compare this grouping with the actual sports, we can see that:

- the jockeys and cyclists have all been allocated to Group 1 (squares)
- the basketball players have all been allocated to Group 2 (circles)
- the rugby players have all been allocated to Group 3 (triangles)
- the footballers and tennis players have been spread amongst the three groups.

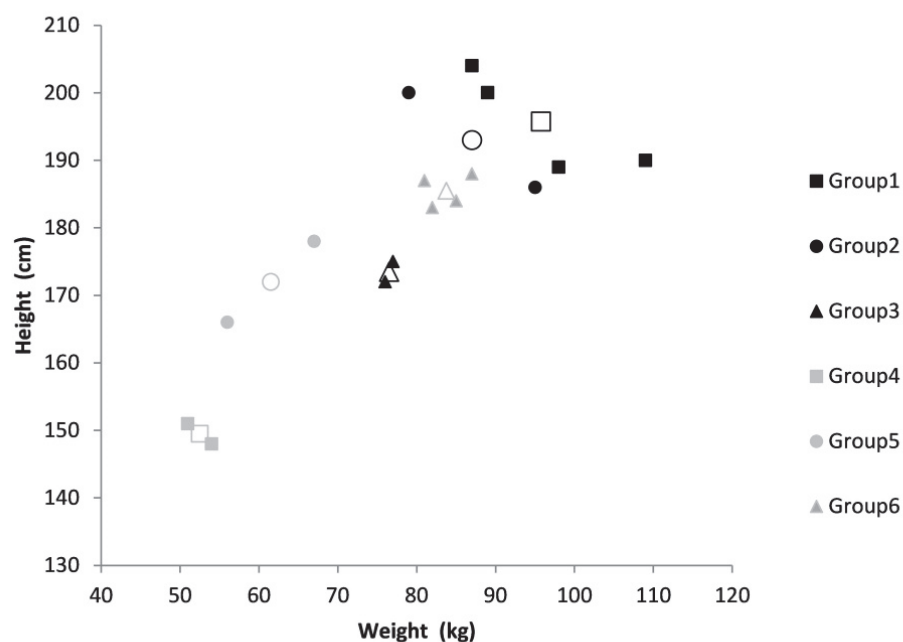
So with $k = 3$ the algorithm has correctly distinguished the sportsmen with more extreme characteristics (eg the very tall basketball players), but was not able to distinguish between the footballers and tennis players, who have quite similar characteristics.

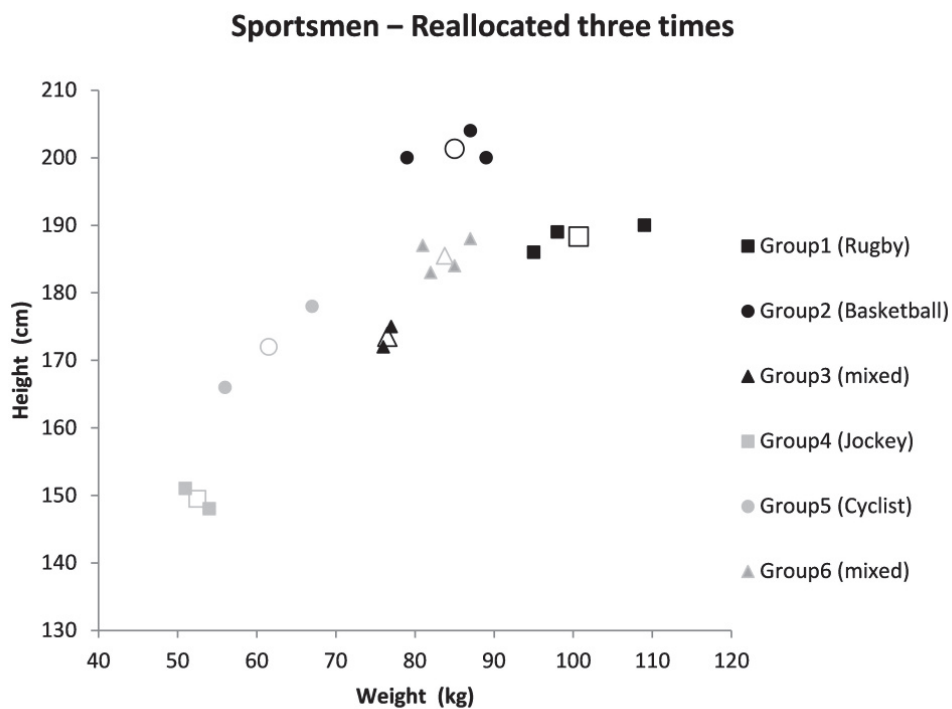
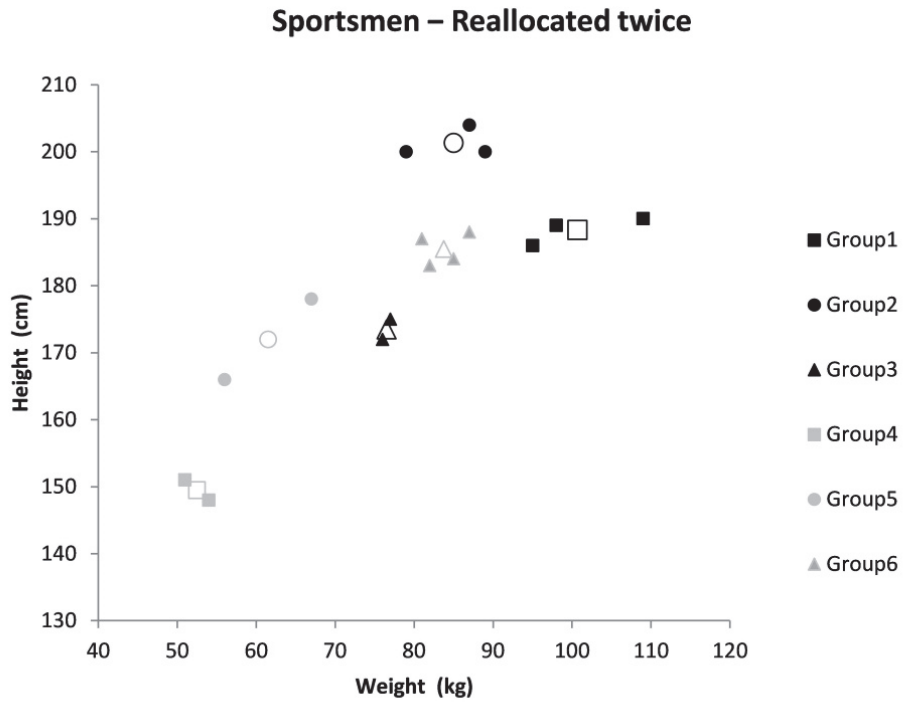
We can try repeating the same steps with $k = 6$ groups (since there were six sports). We've used lighter coloured markers for the three extra groups.

Sportsmen – Random allocations



Sportsmen – Reallocated





There have been no changes, so the algorithm has converged at this stage.

We can see that:

- the jockeys (corresponding to Group 4), cyclists (Group 5), basketball players (Group 2) and rugby players (Group 1) have now all been allocated correctly to separate groups
- the footballers and tennis players have been allocated to Groups 3 and 6, but the split between the two sports is not correct.

So with $k = 6$ the algorithm has performed better, correctly distinguishing all the sportsmen apart from the footballers and tennis players who have quite similar characteristics.

We could now use this final diagram to identify the most likely sport of another sportsman in the competition by checking which of the group centroids his weight and height are closest to.

For example, for a sportsman with weight 65kg and height 185cm, this would be a cyclist.

The table below shows the strengths and weaknesses of the K -means algorithm.

Strengths	Weaknesses
Uses simple principles for identifying clusters which can be explained in non-statistical terms	Less sophisticated than more recent clustering algorithms
Highly flexible and can be adapted to address nearly all its shortcomings with simple adjustments	Not guaranteed to find the optimal set of clusters because it incorporates a random element
Fairly efficient and performs well	Requires a reasonable guess as to how many clusters naturally exist in the data

Source: B. Lantz, *Machine Learning with R* (Birmingham, Packt Publishing, 2013), p. 271

The interpretation and evaluation of the results of K -means clustering can be somewhat subjective. If the K -means exercise has been useful, the characteristics of the clusters will be interpretable within the context of the problem being studied, and will either confirm that the pre-existing opinion about the existence of homogeneous groups has an evidential base in the data, or provide new insights into the existence of groups that were not seen before. One objective criterion that can be examined is the size of each of the clusters. If one cluster contains the vast majority of the cases, or there are clusters with only a few cases, this may indicate that meaningful groups do not exist.

Another way to judge the effectiveness of the algorithm is to repeat the process several times with different random allocations at the start. If similar groupings are obtained each time, it is likely that there is a valid basis underlying the clusters.



R has several machine learning packages that will achieve K -means clustering. One simple one is `kmeans`.

7.2 Principal components analysis

This is covered in Subject CS1.

Principal components analysis is a technique that identifies the dominant combinations of factors that are present in a dataset. The number of such combinations can be chosen so that the model is sufficient to give reasonably accurate results, but not so large that it reflects a lot of the random noise contained in the data.

8 Perspectives of statisticians, data scientists and other quantitative researchers

Machine learning involves the application to data of a range of methods aimed at using data to solve real-world problems. However, many other quantitative researchers would claim to be doing the same thing.

It is certainly true that practitioners from other backgrounds often do work that overlaps with machine learning. Statisticians, for example, do data mining, data reduction using principal components analysis, and routinely estimate logistic regression models. There are differences between the perspectives of many statisticians and that normally adopted by the users of machine learning techniques.

8.1 Terminology

Some of the challenges of communicating with other quantitative researchers are straightforward differences of terminology. In machine learning we talk of ‘training’ a model, or ‘training’ hyper-parameters, whereas statisticians might talk of ‘fitting’ a model or ‘choosing’ higher-level parameters. These are really different words being used for the same activity.

8.2 What is the aim of the analysis?

Some of the differences in perspectives of different groups of researchers are related to the aims of their analyses. This results in interest focusing on different aspects of the models.

This may be illustrated using logistic regression, or discriminant analysis. The logistic regression model may be written:

$$\log\left(\frac{P(y_i = 1)}{P(y_i = 0)}\right) = \beta_0 + \beta_1 x_{1i} + \dots + \beta_J x_{Ji}$$

where y is a binary variable dividing the data into two categories, coded 1 and 0, $x_{1i}, \dots, x_{Ji}$ are the values of the J covariates for case i , and the β ’s are parameters to be estimated from the data.

Statisticians will tend to be most interested in the values and significance of the β ’s, that is in the effect of the covariates on the probability that a case is in either group. They will often present these in tables of *odds ratios*, showing the effect of a difference in the value of a covariate on $\frac{P(y_i = 1)}{P(y_i = 0)}$. Often the purpose of their analyses is to test hypotheses about the effect of a covariate on the odds of y_i being 1.

For example, in a clinical trial, $y_i = 1$ might denote recovery from an illness and $y_i = 0$ denote death, x_1 might be a new drug treatment and $x_2, \dots, x_J$ might be controls. The statistician’s interest is mainly in the size and significance of the parameter β_1 , and especially whether or not β_1 suggests that the new treatment leads to an increase in the odds of recovery. How good the model is at predicting who will recover and who will die is less of an issue.

In machine learning applications, however, the actual values of the β 's are less important than the success of the model in predicting who will recover and who will die, or at discriminating between the two groups (those who recover and those who die). A useful model will be one that makes successful predictions of recovery / death when tested on new data.

Other criteria for assessing the usefulness of models are explicability, and persuading regulators and other supervisory bodies that you have not introduced a classification or discrimination which is perceived as undesirable (for example one based on gender).



In R, there are a wide range of packages which will perform machine learning techniques. This range changes over time. See for example <https://www.r-bloggers.com/what-are-the-best-machine-learning-packages-in-r/> for an overview.

Chapter 21 Summary

What is machine learning?

Machine learning describes a set of methods by which computer algorithms are developed and applied to data to generate information.

Examples can be found in many areas of everyday life, *eg* in targeting online advertising.

Machine learning is increasingly being used in the areas of finance and insurance, *eg* for predicting which borrowers are most likely to default on a loan.

In machine learning, we have a set of features $x_1, x_2, \dots$, each associated with a corresponding output y . The aim is to find a hypothesis, *ie* a function $g(x_1, x_2, \dots)$ that provides a good approximation to the unknown underlying function $y = f(x_1, x_2, \dots)$.

Branches of machine learning

Machine learning algorithms can be divided into:

- supervised learning, where the algorithm is given a set of specific targets to aim for
- unsupervised learning, where the algorithm aims to produce a set of suitable labels
- semi-supervised learning, which involves a combination of supervised and unsupervised learning
- reinforcement learning, where the algorithm aims to improve its performance through repeated use.

Data types

Traditional data analysis in actuarial and statistical applications involves variables of the following types:

- numerical, which may be discrete or continuous
- categorical (*ie* non-numerical), which may be attribute (dichotomous), nominal or ordinal.

However, machine learning often involves data of much more complicated types, *eg* video footage, and the file sizes can be very large.

Concepts and terminology

Loss function

A key element of machine learning is that we need to apply the results to cases not seen in the data used to develop the algorithm. One way to evaluate a hypothesis is to calculate the predictions it makes and to penalise each incorrect prediction by applying a loss function.

Model evaluation

There are various criteria we can use to determine the best model.

Confusion matrices

Classification models that result in Yes or No outputs can be assessed using a confusion matrix for the test set, which shows:

True Positives (TP)	False Negatives (FN)
False Positives (FP)	True Negatives (TN)

We can use this to calculate:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall (sensitivity)} = \frac{TP}{TP + FN} \quad F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These all take values in the range 0% (worst) to 100% (best).

A receiver operating characteristic (ROC) curve can be used to compare models. This plots the true positive rate against the false positive rate.

Generalisation error

An upper bound can be determined for the magnitude of out-of-sample errors. This shows that, with a large enough training set, the out-of-sample error can be made as small as desired.

Train-validate-test

A common approach is to split the available data into three parts:

- a training data set (eg 60%)
- a validation data set (eg 20%)
- a test data set (eg 20%).

Parameters and hyper-parameters

The parameters of a model are variables internal to the model whose values are estimated from the data and are used to calculate predictions using the model.

Hyper-parameters are variables external to the model whose values are set in advance by the user and are chosen based on the user's knowledge and experience to produce a model that works well.

Validation and over-fitting

Machine learning involves a trade-off between bias and variance.

The model should have a sufficient number of parameters to produce accurate results, but not so many that it reflects specific features of the training set that we would not expect to be present in new data.

One way of striking a balance between these two is to apply regularisation or penalisation. This involves working with a loss function of the form:

$$L^*(w_1, w_2, \dots, w_J) + \lambda \sum_{j=1}^J w_j^2$$

Stages of analysis in machine learning

Machine learning tasks can be broken down into the following stages:

- collecting data
- exploring and preparing the data
- feature scaling (to ensure that the input variables have similar ranges of values)
- splitting the data
- training the model
- validation and testing
- improving model performance.

There is a trade-off here between:

- *bias*, ie the lack of fit of the model to the training data, and
- *variance*, ie the tendency for the estimated parameters to reflect the specific data we use for training the model.

It is also important for data analysis to be reproducible and well-documented.

Supervised learning techniques

Penalised generalised linear models

Penalised regression is an adaptation of the method of maximum likelihood where a penalty is applied to constrain the estimated values of the parameters to improve their reliability for making predictions. The method involves maximising the penalised likelihood:

$$\log_e L(\beta_0, \dots, \beta_J \mid x_1, \dots, x_J) - \lambda P(\beta_1, \dots, \beta_J)$$

If the model is parameterised so that the parameter values are expected to be close to zero,

we could use $\sum_{i=1}^J \beta_i^2$ (ridge) or $\sum_{i=1}^J |\beta_i|$ (LASSO) for the penalty function $P(\beta_1, \dots, \beta_J)$.

To select an appropriate number of parameters J (with sample size N), we can minimise:

$$\text{AIC} = \text{Deviance} + 2J \quad (\text{Akaike information criterion})$$

$$\text{or} \quad \text{BIC} = \text{Deviance} + \log N \times J \quad (\text{Bayesian information criterion})$$

Naïve Bayes algorithm

The *naïve Bayes* algorithm uses Bayes' formula to classify items by determining the relative likelihood of each of the possible values for the covariates $x_{1i}, \dots, x_{ji}$. For example, if the observed outcome is $y_i = 1$:

$$P(y_i = 1 \mid x_{1i}, \dots, x_{ji}) \propto P(y_i = 1) \prod_{j=1}^J P(x_{ji} \mid y_i = 1)$$

This method can only be used with categorical covariates and it assumes that the conditional probabilities are independent.

Decision trees (CART analysis)

Decision trees, also known as classification and regression techniques (CART), classify items by asking a series of questions that home in on the most likely classification. It is important to avoid overfitting. Overfitting can be avoided by applying a stopping criterion or by pruning the decision tree. The simplest method of construction is to use greedy splitting.

Gini index for a decision tree

The 'purity' of one of the final nodes in a decision tree can be assessed by calculating the Gini index:

$$G = \sum_{k=1}^m p_k(1 - p_k) = 1 - \sum_{k=1}^m p_k^2$$

where p_k is the proportion of sample items of class k present at that node. This gives a value between 0 ('pure') and 1 ('mixed'). So we would aim to minimise this.

The Gini index for a node where there is a split, or for the whole tree, is:

$$G = \sum_{nodes} \frac{n_{node}}{n} \left(1 - \sum_{k=1}^m p_k^2 \right)$$

where the sum is taken over all the nodes involved and n_{node} is the number of items at the node we are currently evaluating.

Unsupervised learning techniques***K-means clustering***

The k -means clustering algorithm involves modelling the data values as points in space. Starting from an initial cluster allocation (usually random), the method repeatedly finds the centroid of the data points that have been allocated to each cluster and then reallocates the points to the cluster whose centroid they are nearest to. When this process reaches a stage where no further changes are made, the algorithm has converged to the solution.

This method has the advantages that:

- it uses a simple principle that can easily be explained
- it is highly flexible and can easily be adapted to address any shortcomings
- it is efficient and performs well.

However, it also has the disadvantages that:

- it is less sophisticated than more recent clustering algorithms
- it is not guaranteed to find the optimal set of clusters (because of the random element)
- it requires a reasonable guess as to how many clusters naturally exist in the data.

Perspectives

Machine learning techniques are used by statisticians, data scientists and other quantitative researchers, as well as actuaries. However, the different fields often have different aims and also use different terminology.



Chapter 21 Practice Questions

This is a new topic on the actuarial exam syllabus, so as yet the number of past exam questions is limited.

- 21.1**
- (i) Explain the distinction between supervised and unsupervised learning in the context of machine learning.
 - (ii) State whether each of the following applications involves supervised or unsupervised learning or a mixture of the two, and suggest a suitable machine learning algorithm that could be used in each case:
 - (a) predicting a university graduate's salary at age 40 based on the subject they studied, the grade they obtained in their degree and their sex
 - (b) grouping car insurance policyholders into different geographical areas that have similar experience
 - (c) recommending films for subscribers to watch on a subscription movie channel.
 - (iii) Explain the following terms relating to machine learning:
 - (a) hyper-parameters
 - (b) CART
 - (c) greedy splitting
 - (d) clustering.

21.2 A random sample $x_1, \dots, x_n$ of values has been taken from a $N(\mu, \sigma_0^2)$ distribution, where the value of σ_0 is known but the value of μ is unknown. However, it is believed that the value of μ is close to 100. It has been suggested that μ could be estimated using a penalised log-likelihood function.

- (i) Explain the rationale behind this method.
- (ii) Suggest why the penalty function $\lambda(\mu - 100)^2$ would be suitable to use in this case.
- (iii) Show that the estimate of μ derived using this method with the penalty function in part (ii) is:

$$\hat{\mu} = \left(\frac{n}{\sigma_0^2} \bar{x} + 200\lambda \right) / \left(\frac{n}{\sigma_0^2} + 2\lambda \right)$$

- (iv) Comment on how the value of $\hat{\mu}$ calculated using the formula in part (iii) is influenced by the value chosen for the regularisation parameter.
- (v) Explain why this method might be preferable in some circumstances to the basic method of maximum likelihood.

21.3 In a sample of size 100, 10% of individuals have a particular feature.

- (a) Draw up a confusion matrix for a test that can identify this feature perfectly.
- (b) Calculate four measures for the effectiveness of the test, based on the numbers in your matrix.

21.4 A warehouse stores four types of single malt whisky in identical bottles in an underground storeroom, before they are labelled and distributed. Recently the warehouse experienced a major flood in which the stock records were destroyed and the handwritten descriptions on some of the cases were washed off, so that they could no longer be identified.

The warehouse manager has asked you to supervise the process of identifying the type of whisky each of these cases belongs to, so that they can be correctly labelled. A professional whisky taster has provided a report based on a single bottle from each case, which he has classified based on three criteria: Smoky, Fruity, Colour (each on a scale of 1 to 3).

The standard descriptions of the four types are shown in the table below. These can be considered to have a probability of 80% of being correct, while any other description has a probability of 10%. The warehouse manager has also indicated the proportions of each type she believes were in stock at the time of the flood.

Case	Smoky	Fruity	Colour	Proportion in stock
Mactavish	1	3	1	40%
Western Isle	2	1	1	30%
Glenragh	2	2	3	10%
Dogavulin	3	2	2	20%

- (i) Show that, under the assumptions of the naïve Bayes model:

$$P(y = A \mid x_1, x_2, x_3) \propto P(y = A) \times P(x_1 \mid y = A)P(x_2 \mid y = A)P(x_3 \mid y = A)$$

The taster has described the sample bottle from one of the cases as:

Smoky = 2, Fruity = 2, Colour = 2

- (ii) Use the formula from part (i) to estimate how likely this case is to be of each of the four types, and hence recommend how it should be labelled.
- (iii) State two advantages and one disadvantage of the naïve Bayes classification method as a machine learning technique.

- 21.5 A doctor is using k -means clustering with $k = 5$ to classify her patients by height and weight. The raw data shows the patients' statistics in m and kg, but she has converted the heights to cm. She used a method based on Euclidean distance, which converged after 3 iterations, giving the following results:

Group	1	2	3	4	5
Height (cm)	165	160	175	150	185
Weight (kg)	55	65	80	90	100

- (i) Explain what the value of k represents.
- (ii) Explain the reason for the doctor's choice of units.
- (iii) Explain what convergence means in this context.
- (iv) Three new patients have the following data values:

Mr Blobby: (1.64m, 91kg)

Miss Twiggy: (1.87m, 54kg)

Mrs Average: (1.66m, 64kg)

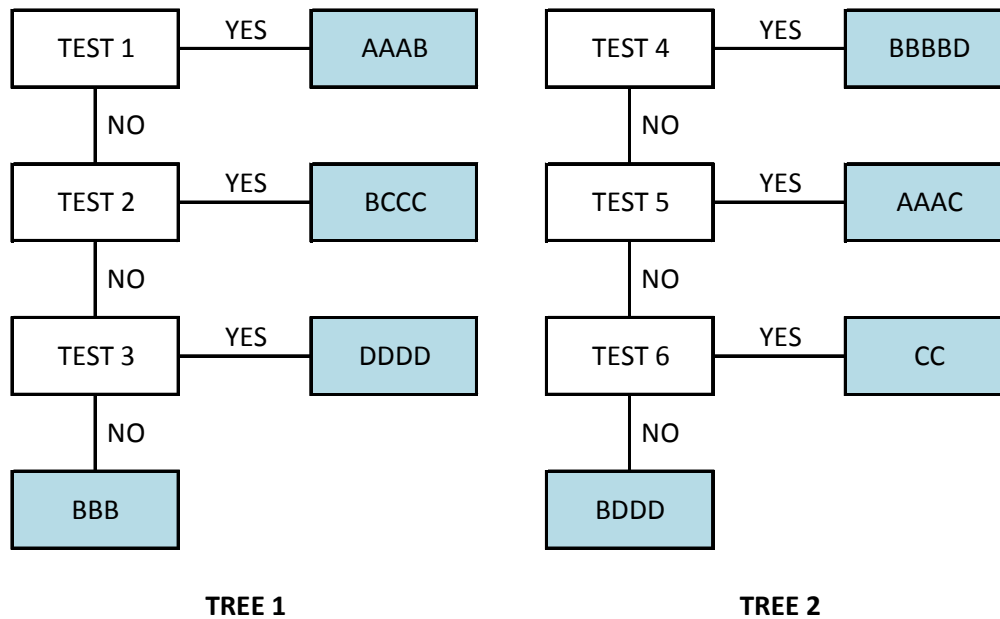
By using a graph, or otherwise, identify the groups to which these patients should be allocated based on their heights and weights.

- (v) State whether the results in part (iv) would have differed if the clusters had been

obtained using the absolute distance metric $D_{abs}(x, k) = \sum_{j=1}^J |x_j - k_j|$.

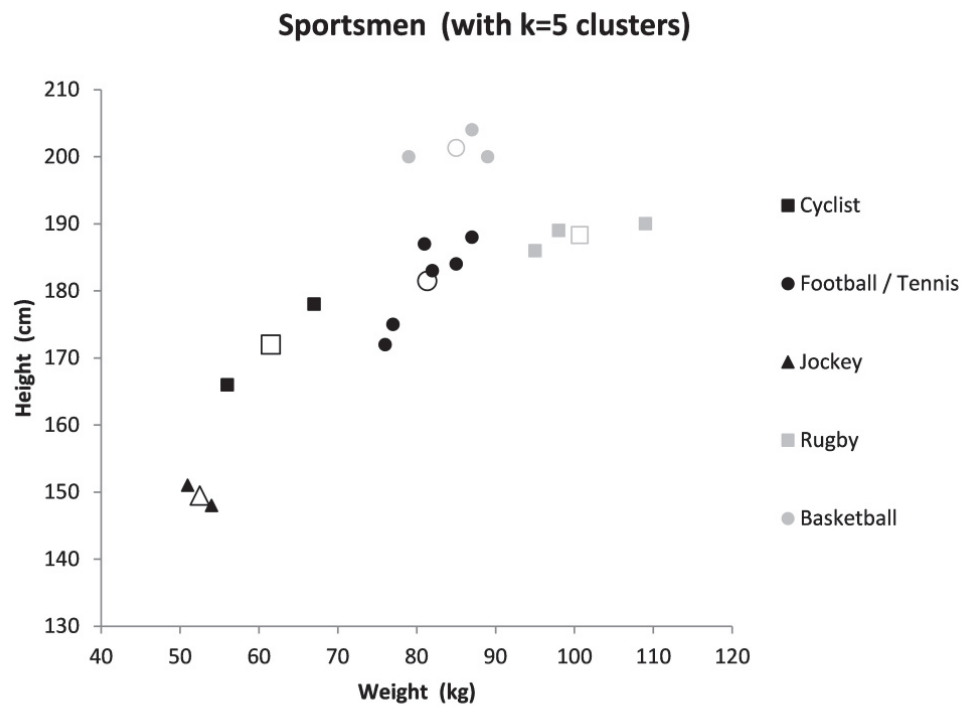
- 21.6 (i) (a) If $p_1 + p_2 + \dots + p_m = 1$, prove the identity $\sum_{k=1}^m p_k \sum_{j=1, j \neq i}^m p_j = 1 - \sum_{k=1}^m p_k^2$.
- (b) Explain how this identity can be used to calculate a measure of the effectiveness of a decision tree.

A researcher is considering two possible decision trees to classify items of four different types labelled A, B, C and D. A sample of 15 items classified using each of the trees gave the results shown below.



- (ii) (a) Calculate the Gini index for each tree.
- (b) Comment on your answers to part (ii)(a).

- 21.7 It has been decided to repeat the k -means method for the sportsmen with $k = 5$, resulting in the following graph:



- (i) Explain why the value $k = 5$ might have been chosen.
- (ii) Use this diagram to identify the most likely sports for three other sportsmen:

Thor (105kg, 200cm), Mo (85kg, 195cm) and Claude (70kg, 165cm)

The solutions start on the next page so that you can
separate the questions and solutions.



Chapter 21 Solutions

21.1 (i) *Supervised and unsupervised learning*

With supervised learning, the desired outcomes for the model are specified in advance and the algorithm aims to reproduce these as closely as possible.

With unsupervised learning, the desired outcomes for the model are not specified in advance and the algorithm aims to assign labels or classify each example in a logical way.

(ii) *Applications*

- (a) Here we would aim to reproduce the salaries for a sample of graduates as closely as possible based on the three variables. So this would be supervised learning. We could use a multiple regression model here.
- (b) Here we would hope that the algorithm can find suitable homogeneous groups that we don't know in advance. So this would be unsupervised learning. We could use the k -means clustering algorithm here.
- (c) Here we would initially make recommendations based on past data from other subscribers but this would be fine-tuned over time based on whether the user accepted or rejected the recommendations. So this would involve a mixture of supervised and unsupervised learning, or a reinforcement algorithm.

(iii) *Terminology*

- (a) As well as the 'internal' parameters that a model estimates from the data and uses to calculate predictions, machine learning methods often also require hyper-parameters, which are external to the model and whose values are set in advance based on the user's knowledge and experience in order to produce a model that works well. An example would be the number of clusters to aim for with the k -means algorithm.
- (b) CART is an abbreviation for classification and regression techniques, which is another name for decision trees. These classify items by asking a series of questions that home in on the most likely classification.
- (c) Greedy splitting is a method of constructing a decision tree. At each stage, we just choose the split that appears to be the most effective at separating the remaining elements, without thinking ahead to the consequences this might have for the later splits.
- (d) Clustering refers to classifying data into a set of homogeneous groups or clusters, which can be done using methods such as the k -means algorithm.

21.2 (i) **Rationale**

This method is based on the method of maximum likelihood where we choose the parameter values to maximise the log-likelihood of the data available. This gives the parameter values that best explain the values in the data.

However, we can also apply a penalty function, which is chosen to make the method more likely to produce parameter estimates close to a set of target values that we are expecting. The penalty is subtracted from the log-likelihood and we maximise this adjusted function instead.

(ii) **Penalty function**

We expect the value of μ to be close to 100. The function $\lambda(\mu - 100)^2$ would be suitable to use here because it takes a large positive value if μ is a long way from 100 (in either direction).

(iii) **Formula**

The likelihood function for the sample is:

$$L = \prod_{i=1}^n \frac{1}{\sigma_0 \sqrt{2\pi}} \exp \left[-\frac{1}{2} \left(\frac{x_i - \mu}{\sigma_0} \right)^2 \right] = \sigma_0^{-n} \exp \left[-\frac{1}{2\sigma_0^2} \sum_{i=1}^n (x_i - \mu)^2 \right] \times \text{constant}$$

So the log-likelihood is:

$$\log L = -n \log \sigma_0 - \frac{1}{2\sigma_0^2} \sum_{i=1}^n (x_i - \mu)^2 + \text{constant}$$

and the penalised log-likelihood is:

$$(\log L)^* = -n \log \sigma_0 - \frac{1}{2\sigma_0^2} \sum_{i=1}^n (x_i - \mu)^2 + \text{constant} - \lambda(\mu - 100)^2$$

To maximise this, we equate the derivative with respect to the parameter μ to zero:

$$\frac{\partial(\log L)^*}{\partial \mu} = +\frac{2}{2\sigma_0^2} \sum_{i=1}^n (x_i - \mu) - 2\lambda(\mu - 100) = \frac{n}{\sigma_0^2} (\bar{x} - \mu) - 2\lambda(\mu - 100) = 0$$

Rearranging this gives:

$$\left(\frac{n}{\sigma_0^2} + 2\lambda \right) \mu = \frac{n}{\sigma_0^2} \bar{x} + 200\lambda$$

So the estimate of μ is:

$$\hat{\mu} = \left(\frac{n}{\sigma_0^2} \bar{x} + 200\lambda \right) / \left(\frac{n}{\sigma_0^2} + 2\lambda \right)$$

(iv) ***Influence of the regularisation parameter***

The regularity parameter λ will be assigned a non-negative value. (Otherwise, it would correspond to a *reward* rather than a penalty.)

If λ was set equal to zero, there would be no penalty and the method would reduce to maximum likelihood estimation. As expected, the formula in part (iii) then gives the usual formula for the MLE of a normal distribution:

$$\hat{\mu} = \left(\frac{n}{\sigma_0^2} \bar{x} + 0 \right) / \left(\frac{n}{\sigma_0^2} + 0 \right) = \frac{n}{\sigma_0^2} \bar{x} / \frac{n}{\sigma_0^2} = \bar{x}$$

If λ is given a very high value, the penalty dominates the calculations and, in the limit, we have:

$$\hat{\mu} = \lim_{\lambda \rightarrow \infty} \left(\frac{n}{\sigma_0^2} \bar{x} + 200\lambda \right) / \left(\frac{n}{\sigma_0^2} + 2\lambda \right) = \lim_{\lambda \rightarrow \infty} \left(\frac{n}{\lambda \sigma_0^2} \bar{x} + 200 \right) / \left(\frac{n}{\lambda \sigma_0^2} + 2 \right) = \frac{200}{2} = 100$$

So now the estimate is equal to the target value of 100.

(v) ***Why this method might be preferable***

The basic (unpenalised) method of maximum likelihood can sometimes lead to unreliable results. The estimated values of the parameters can be very sensitive to the sample data and can vary wildly.

This is most likely to happen when the sample size is small or the likelihood function is very flat so that changes in the parameter values make very little difference to the log-likelihood.

Applying a penalty function encourages the method to produce parameter estimates that are close to the values that would be expected from prior expectations.

21.3 (a) The confusion matrix looks like this:

PREDICTED ACTUAL	YES	NO	TOTAL
YES	TP = 10	FN = 0	10
NO	FP = 0	TN = 90	90
TOTAL	10	90	100

(b) The four measures are:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{10}{10 + 0} = 100\%, \quad \text{Recall} = \frac{TP}{TP + FN} = \frac{10}{10 + 0} = 100\%$$

$$F_1 \text{ score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 100\% \times 100\%}{100\% + 100\%} = 100\%$$

$$\text{False positive rate} = \frac{\text{FP}}{\text{TN} + \text{FP}} = \frac{0}{90 + 0} = 0\%$$

21.4 (i) **Formula for naïve Bayes method**

$$\begin{aligned} P(y = A | x_1, x_2, x_3) &= \frac{P(y = A, x_1, x_2, x_3)}{P(x_1, x_2, x_3)} && \text{(definition of conditional probability)} \\ &= \frac{P(x_1, x_2, x_3 | y = A)P(y = A)}{P(x_1, x_2, x_3)} && \text{(definition of conditional probability in reverse)} \\ &= \frac{P(x_1 | y = A)P(x_2 | y = A)P(x_3 | y = A)P(y = A)}{P(x_1, x_2, x_3)} && \text{(independence assumption)} \\ &\propto P(y = A) \times P(x_1 | y = A)P(x_2 | y = A)P(x_3 | y = A) && \text{(ignore constant factor)} \end{aligned}$$

(ii) **Probabilities for each type**

Let y denote the type and let x_1, x_2, x_3 denote the three descriptions (Smoky, Fruity, Colour).

We can then apply the result from part (i) to calculate the probability that this case is a Mactavish whisky ($y = M$), given that it has been described as Smoky ($S = 2$), Fruity ($F = 2$) and is medium Colour ($C = 2$):

$$\begin{aligned} P(y = M | S = 2, F = 2, C = 2) &\propto P(y = M) \times P(S = 2 | y = M)P(F = 2 | y = M)P(C = 2 | y = M) \\ &= 0.4 \times 0.1 \times 0.1 \times 0.1 = 0.0004 \end{aligned}$$

Similarly for the other three types:

$$\begin{aligned} P(y = W | S = 2, F = 2, C = 2) &\propto P(y = W) \times P(S = 2 | y = W)P(F = 2 | y = W)P(C = 2 | y = W) \\ &= 0.3 \times 0.8 \times 0.1 \times 0.1 = 0.0024 \end{aligned}$$

$$\begin{aligned} P(y = G | S = 2, F = 2, C = 2) &\propto P(y = G) \times P(S = 2 | y = G)P(F = 2 | y = G)P(C = 2 | y = G) \\ &= 0.1 \times 0.8 \times 0.8 \times 0.1 = 0.0064 \end{aligned}$$

$$\begin{aligned} P(y = D | S = 2, F = 2, C = 2) &\propto P(y = D) \times P(S = 2 | y = D)P(F = 2 | y = D)P(C = 2 | y = D) \\ &= 0.2 \times 0.1 \times 0.8 \times 0.8 = 0.0128 \end{aligned}$$

So the probabilities for each type are in the ratio:

$$M : W : G : D = 4 : 24 : 64 : 128 = \frac{4}{220} : \frac{24}{220} : \frac{64}{220} : \frac{128}{220} = 0.0182 : 0.1091 : 0.2909 : 0.5818$$

So the recommendation would be that this case is a Dogavulin whisky, as this has a far higher probability (58%) than the other three types.

(iii) ***Advantages and disadvantages***

Advantages of the naïve Bayes method include:

- it is easy to apply
- it requires very little data.

The main disadvantage is that it assumes that the conditional probabilities are independent (which can be a poor approximation when the variables are correlated).

21.5 (i) ***Meaning of k***

k is a hyper-parameter specifying the number of clusters the algorithm should aim to produce – in this case, 5.

(ii) ***Choice of units***

The weights in the original data provided were given in units of kilograms. These cover a range of values of about 50kg.

The heights in the original data provided were given in units of metres. These cover a range of values of about 0.50m.

So with units of (kg, m) the range for the weights is about 100 times greater, which would mean that the weights would totally dominate the calculations and the heights would effectively be ignored.

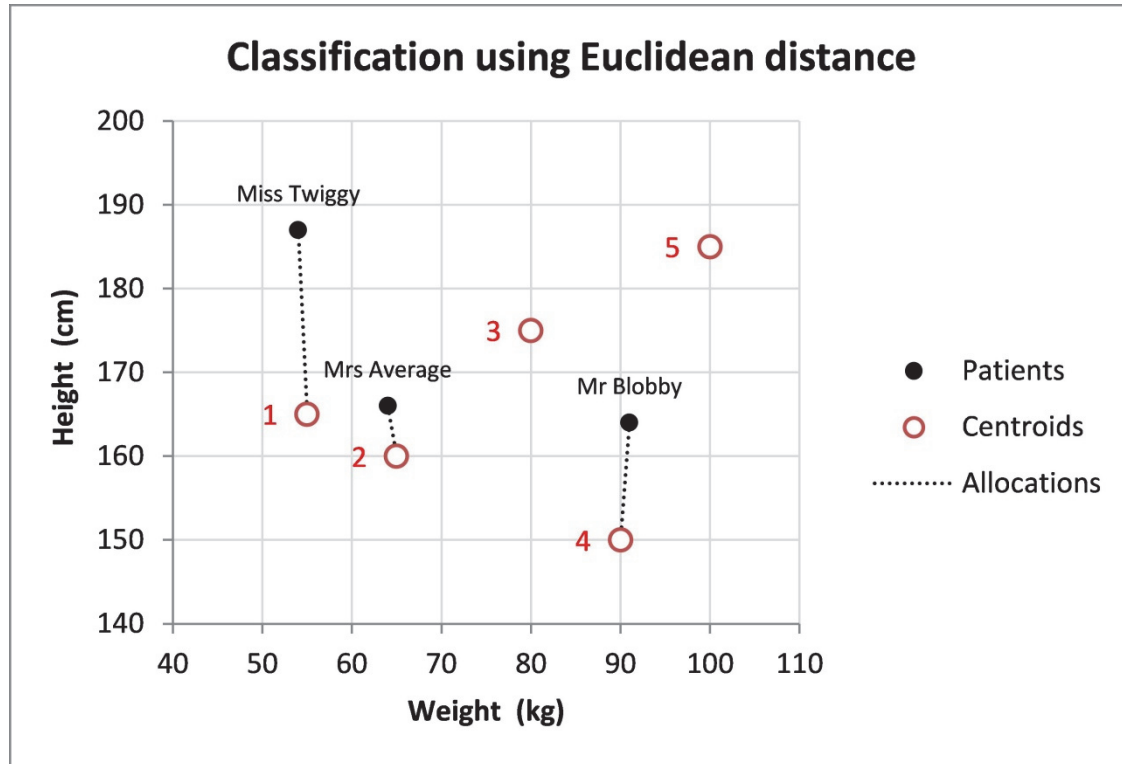
However, when the doctor converts the heights to centimetres, the range of values is then about 50cm, which is numerically very similar to the range for the weights. This gives the two variables a similar weighting in the calculations.

(iii) ***Convergence***

The algorithm involves repeatedly finding the centroid of the data points that have been allocated to each cluster and then reallocating the points to the cluster whose centroid they are nearest to. When this process reaches a stage where no further changes are made, the algorithm has converged to the solution.

(iv) **Classification of new patients**

If we plot the patients and the centroids for the clusters on a graph, we can easily see which centroid each patient is nearest to, and hence classify these patients.



If we do the calculations, we find that the shortest Euclidean distances D_i (for the centroids $i = 1, 2, \dots, 5$) are:

$$\text{Mr Blobby: } (1.64\text{m}, 91\text{kg}) \rightarrow D_4 = \sqrt{(164 - 150)^2 + (91 - 90)^2} = 14.04$$

$$\text{Miss Twiggy: } (1.87\text{m}, 54\text{kg}) \rightarrow D_1 = \sqrt{(187 - 165)^2 + (54 - 55)^2} = 22.02$$

$$\text{Mrs Average: } (1.66\text{m}, 64\text{kg}) \rightarrow D_2 = \sqrt{(166 - 160)^2 + (64 - 65)^2} = 6.08$$

(v) **Absolute distance**

The absolute distance measures the distance between points assuming that we can only move horizontally or vertically.

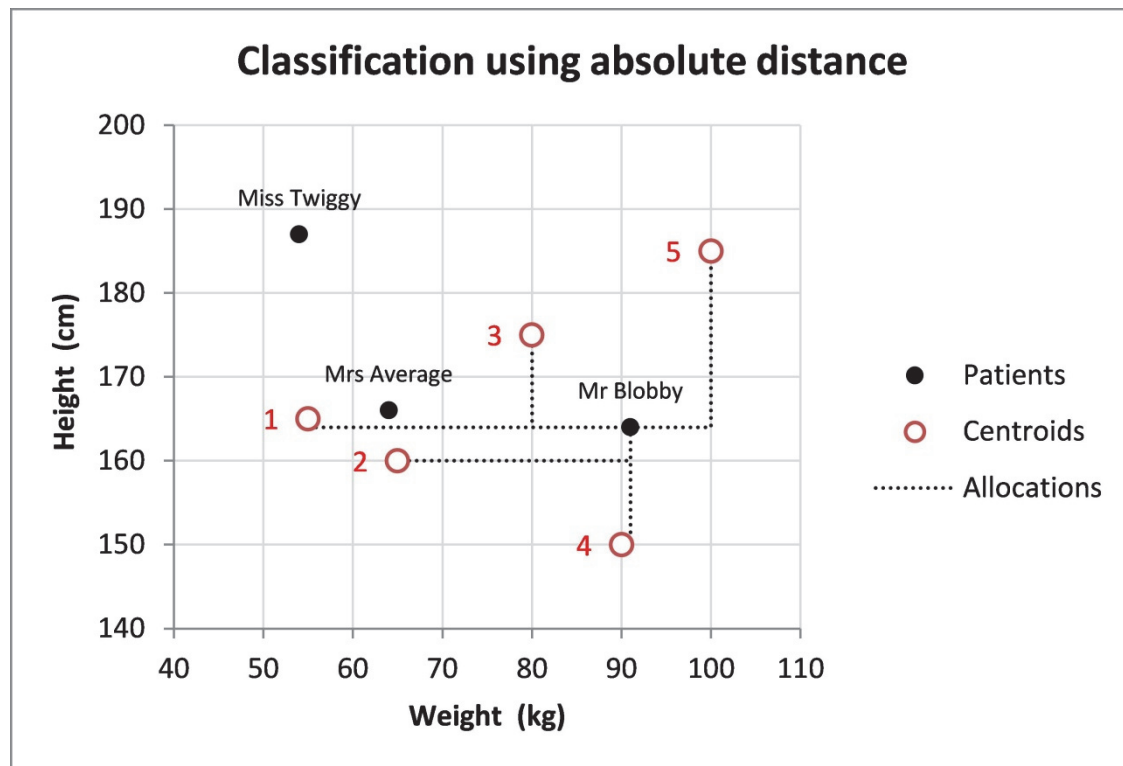
Here we have two dimensions (the two variables Weight and Height), so $J = 2$. With this metric the distance between the two points (x_1, x_2) and (k_1, k_2) is:

$$|x_1 - k_1| + |x_2 - k_2|$$

With this metric, Mr Blobby's distance from the centroid for cluster 4 is:

$$\text{Mr Blobby: } (1.64\text{m}, 91\text{kg}) \rightarrow D_4 = |164 - 150| + |91 - 90| = 14 + 1 = 15$$

The diagram below shows the distance to each centroid for Mr Blobby.



We can see that using absolute distances would give the same answers as Euclidean distance for all three of these patients.

21.6 (i)(a) **Prove the identity**

$$\sum_{k=1}^m p_k \sum_{j=1, j \neq i}^m p_j = \sum_{k=1}^m p_k (1 - p_k) = \sum_{k=1}^m (p_k - p_k^2) = \sum_{k=1}^m p_k - \sum_{k=1}^m p_k^2 = 1 - \sum_{k=1}^m p_k^2$$

(i)(b) **Measuring the effectiveness of a decision tree**

We can measure the effectiveness of a decision tree by examining the 'purity' of the groupings produced for a training data set.

This can be done by multiplying the proportion of items of type k at each node by the proportions for each other type $j \neq k$ and summing. These values are then weighted by the number of items at that node to calculate an overall measure called the Gini index. Using the identity above leads to the following formula:

$$G = \sum_{nodes} \frac{n_{node}}{n} \left(1 - \sum_{k=1}^m p_k^2 \right)$$

where the sum is taken over all the nodes and n_{node} is the number of items at the node we are currently evaluating.

(ii)(a) **Calculate the Gini index**

In Tree 1 the top final node contains AAAB, ie 3 A's and 1 B. So the proportions are $p_1 = \frac{3}{4}$ and $p_2 = \frac{1}{4}$. So the Gini index for this node is:

$$G = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = \frac{3}{8}.$$

The next node contains BCCC, ie 3 C's and 1 B. So it also has a Gini index of $\frac{3}{8}$.

The other two final nodes are 'pure', as each contains a single label. So they have a Gini index of 0.

To find the Gini index for the whole of Tree 1, we need to work out the weighted average of the Gini index at each of the final nodes weighted by the number of elements they contain, ie 4, 4, 4, and 3 (making a total of 15). So this gives:

$$G = \left(\frac{4}{15} \times \frac{3}{8}\right) + \left(\frac{4}{15} \times \frac{3}{8}\right) + \left(\frac{4}{15} \times 0\right) + \left(\frac{3}{15} \times 0\right) = \frac{1}{5} = 0.2$$

Similarly, for Tree 2 the Gini index is:

$$\begin{aligned} G &= \frac{5}{15} \left[1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 \right] + \frac{4}{15} \left[1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 \right] + \frac{2}{15} [1 - 1^2] + \frac{4}{15} \left[1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 \right] \\ &= \left(\frac{5}{15} \times \frac{8}{25}\right) + \left(\frac{4}{15} \times \frac{3}{8}\right) + 0 + \left(\frac{4}{15} \times \frac{3}{8}\right) = \frac{23}{75} = 0.307 \end{aligned}$$

(ii)(b) **Comment**

The Gini index for Tree 1 is lower than for Tree 2. So, using this criterion, Tree 1 would be preferred.

21.7 (i) *Choice of value of k*

The graph with $k = 6$ correctly identified the jockeys, cyclists, basketball players and rugby players into logical groups. However, the footballers and tennis players' characteristics are too similar for us to be able to distinguish them effectively using this approach. So a sensible approach is to combine the footballers and tennis players into a single group and look for 5 logical clusters.

(ii) *Identify the sports*

In each case we allocate each sportsman to the group whose centroid he is closest to. This gives:

Thor = Rugby, Mo = Basketball, Claude = Cyclist

End of Part 5

What next?

1. Briefly **review** the key areas of Part 5 and/or re-read the **summaries** at the end of Chapters [17](#) to [21](#).
2. Ensure you have attempted some of the **Practice Questions** at the end of each chapter in Part 5. If you don't have time to do them all, you could save the remainder for use as part of your revision.
3. Attempt **Assignment X5**.

Time to consider ...

... 'rehearsal' products

Mock Exam and Marking – You can attempt the Mock Exam and get it marked. Results of surveys have found that students who do a mock exam of some form have significantly higher pass rates. Students have said:

'I find the mock a useful tool in completing my pre-exam study. It helps me realise the areas I am weaker in and where I need to focus my study.'

'Overall the marking was extremely useful and gave detailed comments on where I was losing marks and how to improve on my answers and exam technique. This is exactly what I was looking for - thank you!'

You can find lots more information on our website at www.ActEd.co.uk.

Buy online at www.ActEd.co.uk/estore

And finally ...

Good luck!